FOUNDATIONS OF SOFTWARE ENGINEERING

Why building software systems is not the same as writing code

WHAT SOFTWARE ENGINEERING IS REALLY ABOUT

Not “just writing more code” — building useful systems under real-world constraints

Software engineering = finding real solutions to real problems, on actual schedules, with available resources.

Build for people

A useful system solves a problem for users, customers, colleagues, or society. Working code without real value is not enough.

Build within constraints

Deadlines, budgets, team size, safety, regulation, performance, maintainability, and uncertainty shape the engineering work.

Build for change

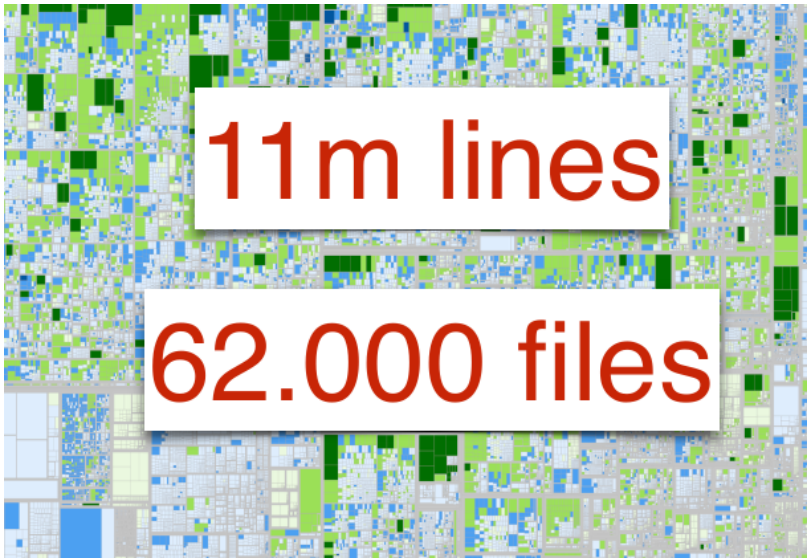
Real software is not finished when it first runs. It must survive new requirements, bugs, integrations, and changing environments.

The key shift from “programming” to “software engineering” is: the system must work in the world, not only on your laptop.

A MILLION-LINE SYSTEM IS NOT JUST “MORE CODE”

Large-scale difficulty comes from interaction, coordination, and change — not arithmetic

1,000 × 1,000-line programs ≠ one 1,000,000-line software system



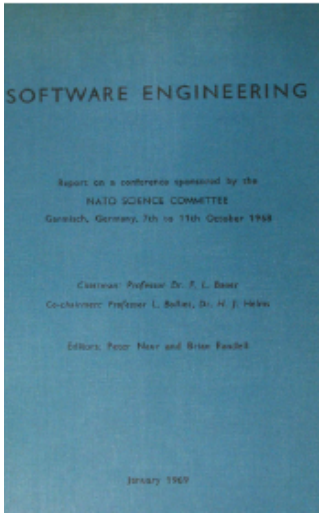
Why the “1000× more code” intuition fails

- Interfaces: parts must agree on protocols, assumptions, and data formats
- Coordination: many people and teams must understand what others are doing
- Integration: independent pieces must still behave as one coherent system
- Deployment: the system must run, update, recover, and operate in production
- Evolution: the structure must survive years of change without falling apart

Large systems are hard because relationships grow faster than lines of code do.

WHY A DISCIPLINE HAD TO EMERGE

The “software crisis” was really a crisis of complexity, expectations, and change



Complexity

Programs became orders of magnitude larger and more interconnected than what teams had built before.

Expectations

Hardware got cheaper and more powerful, so people expected software to keep up — but software cost and difficulty kept rising.

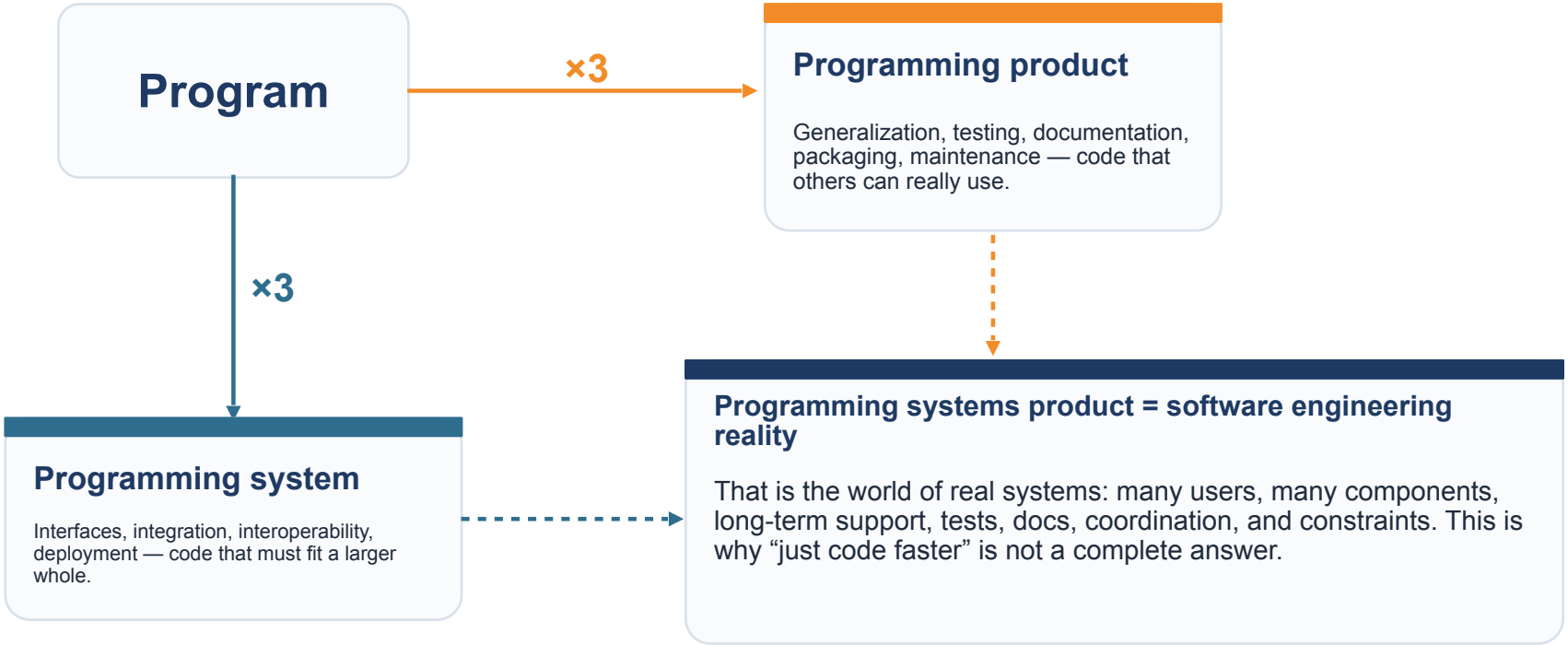
Change

Systems had to be maintained, corrected, extended, and adapted after first release. That turned out to be normal, not exceptional.

1968: “software engineering” gained visibility because building large software stopped looking like isolated programming and started looking like serious systems work.

FROM “PROGRAM” TO “SOFTWARE”

Brooks’s key idea: once code must serve others and fit a system, the work multiplies



THE TAR PIT OF LARGE-SYSTEM BUILDING

Any one difficulty is manageable. The combination is what traps teams.



Brooks's warning

Large-system programming has long been a tar pit: many powerful teams emerge with running systems, but far fewer meet goals, schedules, and budgets. The trap is not one monster problem; it is the accumulation of simultaneous, interacting factors.

Scope

More goals than the team can absorb.

Coupling

Every part affects several others.

Change

The target moves while you build.

The mission of software engineering is still the one defined by Brooks: real solutions to real problems, on actual schedules, with available resources.

WHAT WE STUDY IN THIS COURSE

Software engineering combines process, technique, and tool support

Processes

How work is organized over time: phases, iterations, feedback loops, planning, coordination, and delivery.

Techniques

How we approach specific problems: eliciting requirements, modeling, testing, design, estimation, and analysis.

Tools

How we automate and support the work: version control, CI/CD, test automation, issue tracking, and now AI assistance.

An engineering discipline concerned with all aspects of software production: from deciding what to build to keeping it working over time.

SOFTWARE IS ENGINEERED, NOT MANUFACTURED

For software, most cost and quality problems are created before the first copy is made

Hardware

1. Engineer the design
2. Manufacture physical units
3. Manufacturing can introduce wear, defects, and variability
4. Broken parts can often be replaced with spare parts

Software

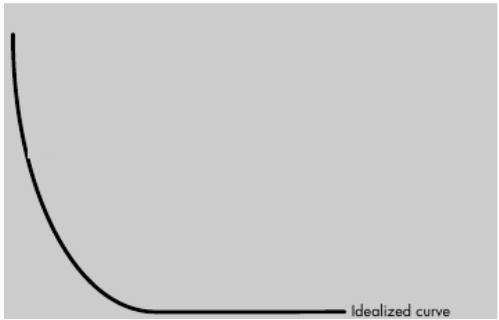
1. Analyze, design, implement, test
2. Copying the software is almost free
3. “Manufacturing” does not usually create new quality problems
4. Failures usually trace back to requirements, design, code, or later changes

Software projects cannot be managed as if they were manufacturing projects.

SOFTWARE DOES NOT “WEAR OUT” — IT DETERIORATES

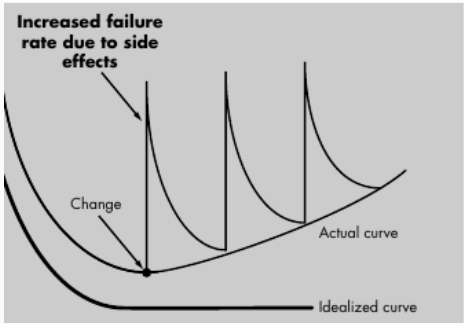
Parnas and Pressman: the main cause is repeated change

Ideal case



If we only fixed defects and introduced no new ones, failure rate would fall and then stay low.

What really happens



New changes, fixes, integrations, and adaptations create side effects. The system’s “minimum failure level” slowly rises unless teams actively control complexity.

Software aging is about accumulating change, not physical wear.

WHY SOFTWARE AGES

The real enemy is repeated change under pressure

Parnas's core idea: programs get old because they keep changing — and every change touches understanding, structure, and assumptions.

Enhance

New business rules, new features, and new user expectations change what the system must do.

Adapt

New platforms, new environments, new integrations, and new security demands change where and how it must run.

Repair under uncertainty

Fixes made quickly, with incomplete understanding, can improve today's bug while making tomorrow's system harder to change.

There are no "software spare parts":
every failure points back to design, code, or the process that changed them.

LEHMANN'S LAWS: EVOLUTION IS NORMAL

Large software does not stay useful by standing still

Continuing change

A useful system must keep adapting. If it stops changing while the world changes, it becomes progressively less satisfactory.

Increasing complexity

As a system evolves, complexity tends to rise — unless teams deliberately do work to control and reduce it.

Conservation of familiarity

Growth is constrained by how much the people involved can still understand. Teams cannot safely absorb unlimited change.

This last law matters even more today:
AI can generate change faster than teams can truly understand it.

INTERMEZZO: AI CHANGES THE RATE OF CHANGE

CircleCI 2026 + DORA 2025/2026 + the return of Lehmann's "familiarity" problem

> *Teams are producing more code than ever, but fewer changes actually reach customers. Across CircleCI projects, average throughput rose 59% year over year, while the median team saw main-branch throughput fall 7%. DORA reports that time saved in creation is often re-spent on auditing and verification.*
>
> — CircleCI (2026) + DORA (2026)

AI helps creation

Code generation and agent workflows raise local throughput.

The bottleneck moves

Validation, review, recovery, integration, and deployment absorb the real cost.

Familiarity matters

Teams can now change more than they can comfortably understand. That is Lehmann's law, updated.

FROM VIBE CODING TO AUGMENTED CODING

AI should amplify engineering discipline, not replace it!

Vibe coding

- Treat AI as a magic code generator
- Start from vague intent
- Trust plausible output too easily
- Discover mismatches late
- Often increases regressions and maintainability risk

Augmented coding

- Use AI as a productivity multiplier
- Start from clearer goals, constraints, and examples
- Keep tests, review, and architecture thinking
- Verify incrementally
- Preserve core engineering discipline

The difference is not whether AI is used.
The difference is whether engineering judgment stays in charge.

THE QUEST FOR HIGHER ABSTRACTION LEVELS

Software engineering keeps moving work upward



Grace Hopper’s era

Compilers let people describe intent at a higher level instead of writing everything in machine terms.

Booch’s view

The history of the field is a rise in abstraction: from algorithms, to design abstractions, to systems and platforms.

Fowler’s update

LLMs raise the abstraction level again — but with non-deterministic tools, so checks and guardrails matter more.

REUSE KEEPS RISING — BUT THE SYSTEM IS STILL CUSTOM

We reuse more components, yet the whole product still needs engineering

Modern engineering increasingly means: compose more, integrate more, and reason about boundaries more.



Good news

We build more from proven components. That lets engineers spend more energy on higher-value decisions.

Trade-off

Every reused piece also introduces interfaces, assumptions, updates, security concerns, and integration work.

Big lesson

Reuse reduces how much code we write by hand. It does not eliminate the engineering needed to make the whole system fit.

THE MAIN COST OF SOFTWARE ENGINEERING WAS NEVER CODING

Brooks's scheduling rule still shocks because it makes coding look small



Planning is bigger than many expect

Requirements, architecture, coordination, and risk management consume real time before coding starts.

Testing is half the schedule

Bad news discovered late is expensive. Rework in system testing or after release dominates cost and schedule pain.

Coding is only one slice

Fast code generation does not solve weak requirements, poor design, brittle integration, or unstable delivery pipelines.

INTERMEZZO: ABUNDANT CHEAP CODE, SCARCE UNDERSTANDING

Kent Beck's challenge for the AI era

> *As code gets cheaper, what becomes scarce is understanding, judgment, the ability to see how pieces fit together, and the wisdom to know what not to build.*
>
> **— Kent Beck (2025), paraphrased**

Understanding

What problem are we really solving, and what constraints shape it?

Judgment

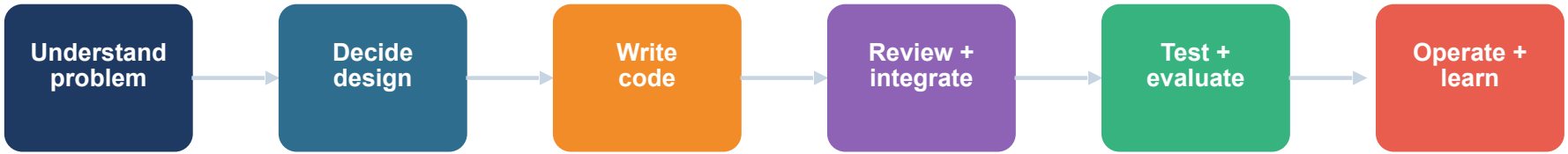
Which generated solution is acceptable, safe, maintainable, and worth keeping?

System fit

How do the pieces fit into architecture, operations, and long-term evolution?

CODING HAS NEVER BEEN THE GOVERNING BOTTLENECK

Once software becomes non-trivial, progress is dominated by understanding, coordination, and feedback



Local speed vs system speed

You can type faster or generate more code without moving the overall system much.

Late defects hurt most

Cost and schedule are driven mainly by rework once problems surface in integration, testing, or production.

AI makes this visible

Cheap code generation exposes the real bottlenecks instead of removing them.

BROOKS' LAW

Adding manpower to a late software project makes it later

3 people × 4 months ≠ 6 people × 2 months

Onboarding time

New people need context: goals, architecture, conventions, data, tools, and the current state of the effort.

Communication overhead

More people create more coordination channels, more meetings, more reviews, and more opportunities for misunderstanding.

Sequential constraints

Some work simply cannot be parallelized away. Certain discoveries must happen before other work can proceed.

Brooks's law is really a claim about system structure, not a complaint about people.

WHY BROOKS' LAW HAPPENS

Software has communication and dependency overhead

Partitionable tasks

Some work can be split fairly well. Adding people helps — but software rarely looks like simple harvesting.

Sequential tasks

Other work must happen in order. Extra people do not remove the waiting.

Communication-heavy tasks

If people must constantly synchronize, the gain is poorer than an even trade of people for months.

Tightly coupled tasks

Complex interrelationships can fully counteract the division of the work.

Communication channels grow as N(N−1)/2

2 people
1 channel

4 people
6 channels

6 people
15 channels

10 people
45 channels

INTERMEZZO: DO AI AGENTS BREAK BROOKS' LAW?

The tooling changed. The system-level constraints did not disappear.

> *Organizations are constrained by human and systems-level problems. We remain skeptical of the promise of any technology to improve organizational performance without first addressing human and systems-level constraints.*
>
> **— Kent Beck, Laura Tacho, and Steve Yegge (2026)**

Agents write faster

Yes: more code, more PRs, more experiments, more local throughput.

But review + QA remain

Validation, integration, and recovery still limit what safely reaches users.

So Brooks still matters

The overhead is relocated, not eliminated: context, contracts, tests, evals, ownership.

AI RELOCATES THE OVERHEAD

Cheap code makes some old work smaller — and makes other work more important

Context engineering

Give the agent the right repo guidance, examples, rules, and local tools. Too little context causes drift; too much creates noise.

Real contracts

Interfaces must specify what goes in, what comes out, what must hold, and what is never allowed.

Verification + evals

Generated output must be checked ruthlessly: tests, CI, observations, acceptance checks, and post-release telemetry.

Human judgment

Humans steer, set boundaries, and decide whether the outcome is actually acceptable.

Rigor does not disappear; it moves closer to reality: into specs, contracts, tests, monitoring, and guardrails.

OLD MYTHS THAT ARE STILL WRONG IN 2026

AI changes the surface of the work. It does not magically remove engineering reality.

Myth 1

“Sketchy requirements are enough to start; we can figure out the rest later.”

Reality: ambiguity becomes more expensive when AI scales wrong assumptions quickly.

Myth 2

“If software is flexible, requirements can always change cheaply.”

Reality: repeated change without discipline causes deterioration and rework.

Myth 3

“Once the program works, the job is done.”

Reality: shipping, operating, supporting, and evolving the system are part of the job.

Myth 4

“More generated code means more value delivered.”

Reality: delivered value depends on integration, testing, reliability, and fit with real needs.

WHAT GROWS, SHRINKS, AND CHANGES FOR STUDENTS

The tools are changing, so the leverage shifts too

GROWS

- Problem framing
- Requirements and constraints
- System design and architecture
- Verification, tests, and evals
- Trade-off reasoning
- Coordination and communication
- Critical review of AI output

SHRINKS

- Hand-writing boilerplate
- Memorizing syntax trivia
- Drawing boilerplate diagrams by hand
- Mistaking code quantity for progress
- Treating “typing speed” as the core professional skill

CHANGES

- Docs become living context for humans + tools
- Coding becomes more supervisory
- Quality shifts toward explicit checks and harnesses
- Junior engineers need broader system awareness earlier

Brooks’s law is really a claim about system structure, not a complaint about people.

WHY THIS CHAPTER COMES BEFORE SDLC

Once you see the constraints, the need for process becomes obvious

Complexity

Large systems are about coordination and integration, not only code.

Change

Software ages. Evolution is normal.

Late defects

Testing and rework dominate cost.

Coordination

Brooks’s law punishes naive staffing fixes.

All of this leads to the next question:
how should teams organize the work over time?

SOFTWARE DEVELOPMENT PROCESSES

From Waterfall to AI-Native Development

TWO FUNDAMENTAL PROCESSES

Everything else is a variant of these two approaches

There are only **TWO** fundamental ways to organize software development

WATERFALL

Breaks down project by ACTIVITIES
(Requirements → Design → Code → Test)

ITERATIVE

Breaks down project by FUNCTIONALITY
(Feature 1 → Feature 2 → Feature 3...)

WHY DID WATERFALL EMERGE?

Understanding the 1960s-70s Context

> *"Early structured methodologies were very waterfall-ish, because **the risks of failure were very high**. Machine time was expensive. You had to plan."*
>
> — Grady Booch

Expensive Compute

Machine time cost more than programmer time. Every CPU cycle had to be justified.

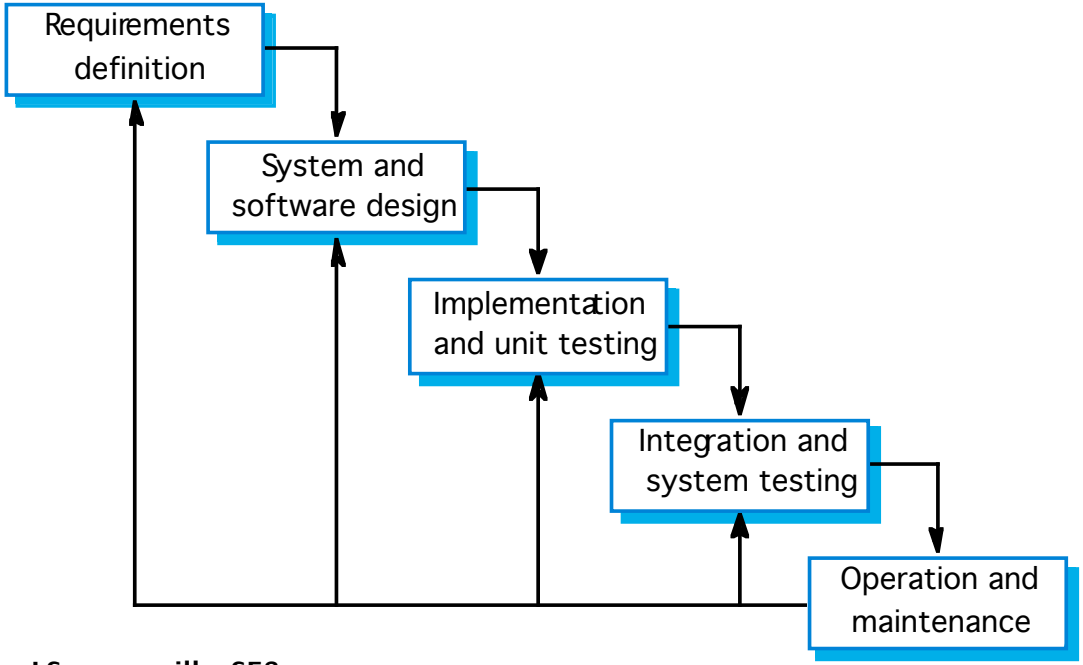
No "Undo" Button

Punch cards, batch processing. Mistakes were costly. You planned everything upfront.

Large Organizations

Military, aerospace, banking. Formal processes matched organizational culture.

THE WATERFALL MODEL



I.Sommerville, SE8

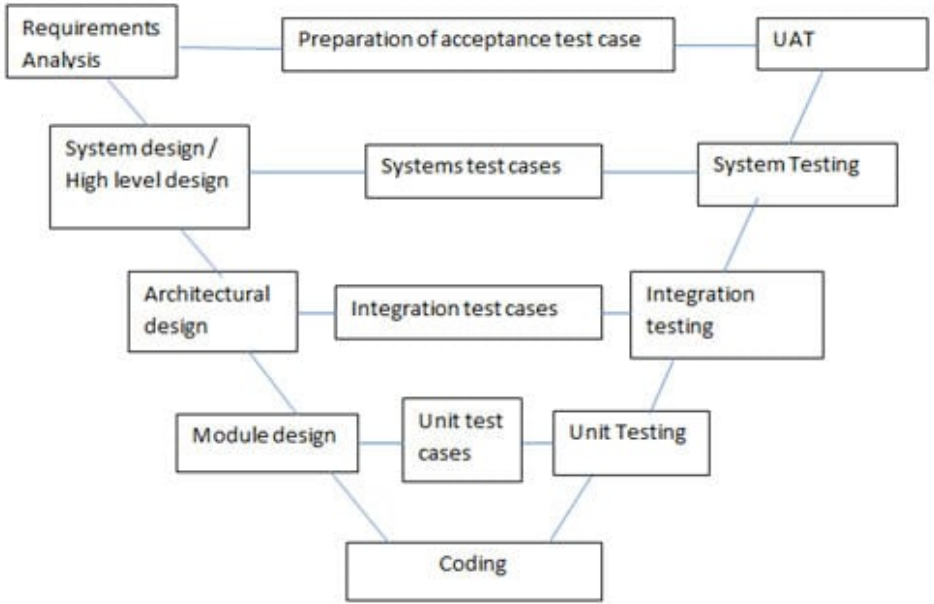
KEY CHARACTERISTICS

- ✓ Sequential phases
- ✓ Complete before moving on
- ✓ Extensive documentation
- ✓ Predictive planning

- ✗ Difficult to accommodate change
- ✗ Late discovery of problems
- ✗ Customer sees product late

THE V-MODEL

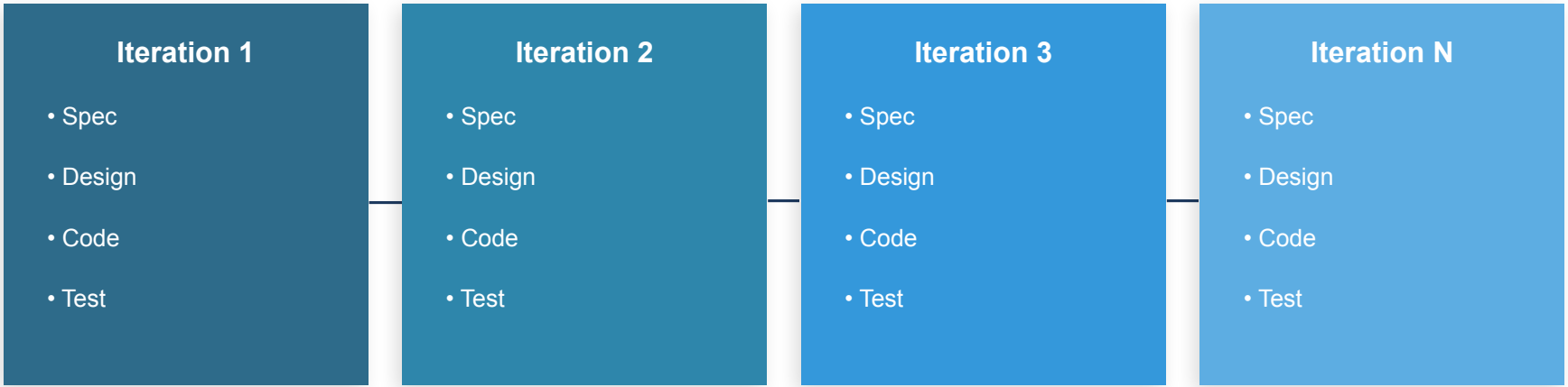
A Waterfall Variant with Testing Focus



Each development phase has a corresponding testing phase. Test planning happens early!

ITERATIVE DEVELOPMENT

The paradigm that follows Adaptive Planning



CORE PRINCIPLE:
Each iteration delivers a **REAL, WORKING PRODUCT,** not a prototype, not a draft!

Benefits:

• Early value delivery • Continuous feedback • Lower risk • Adapt to change • Highest priority features tested most

TIME BOXING

The Discipline That Makes Iterative Development Work

THE GOLDEN RULE

When time runs out... **SLIP FEATURES, NOT DATES**

FIXED

Iteration duration

Release dates

Team capacity

VARIABLE

Feature scope

What gets included

Lower priority items

This forces prioritization... That's a feature, not a bug!

PREDICTIVE vs ADAPTIVE

Two Philosophies of Planning

PREDICTIVE PLANNING

- Make **detailed plans upfront**
- **Follow strictly** the plan strictly
- **Changes are deviations**/anomalies
- Fixed scope, fixed price

Assumes: *Requirements are stable and knowable*

ADAPTIVE PLANNING

- Plan is a baseline, not a contract
- Continuous **re-planning**
- Welcome **change**!
- Fixed price, variable scope

Assumes: *Software is controllable but not predictable*

*"Plans are useless, but **planning is indispensable**."*
Dwight D. Eisenhower

Prediction is an illusion — there is no 'according to plan' in adaptive planning, but there IS planning!

THE BREAKING POINT

Why Waterfall and traditional Iterative couldn't keep up (1990s-2000s)

The DOT COM era (1999-2000): An **18-month delivery cycle** was no longer acceptable. *The business demanded faster feedback loops.*

Context from Kent Beck's on Pragmatic Engineer Podcast (Jun 2025)

Time-to-Market

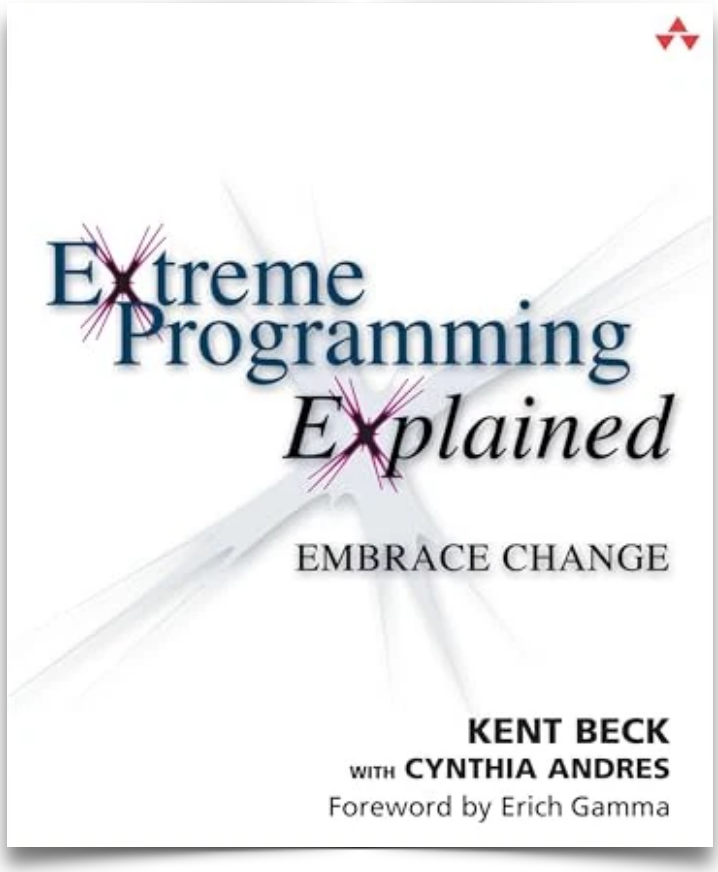
Internet startups needed to ship in weeks, not years

Changing Requirements

Markets moved faster than specs could be written

Cheaper Computing

Machine time became cheap; iteration became possible



- **Test-Driven Development (TDD):** Developers write automated unit tests *before* writing the production code. The code is only considered complete once it passes these tests.
- **Pair Programming:** Two developers work together at a single workstation: one "driver" writes code while the "observer" reviews it in real-time to catch errors and share knowledge.
- **Continuous Integration (CI):** Teams integrate their code changes into a shared repository multiple times a day, triggering automated builds and tests to detect issues early.
- **Refactoring:** Developers continuously improve the internal structure of the code without changing its external behavior to keep the codebase clean and maintainable.
- **Small Releases**: Delivering functional software in short cycles (every 1–3 weeks) to provide value and gather customer feedback as soon as possible.

THE AGILE MANIFESTO

Snowbird, Utah — February 2001

Individuals & interactions	over	processes & tools
Working software	over	comprehensive documentation
Customer collaboration	over	contract negotiation
Responding to change	over	following a plan

"While there is value in the items on the right, we value the items on the left more."

AGILE PRINCIPLES (HIGHLIGHTS)



Deliver working software frequently



Welcome changing requirements



Cross-functional, self-organizing teams



Maintain a sustainable pace



Continuous attention to technical excellence



Regular reflection and adaptation

Agile is about VALUES and PRINCIPLES, not specific practices!

*We threw away long-range plans. We **rejected heavyweight processes**. We stopped pretending we could predict the shape of a system months in advance. We paired constantly, which looked inefficient. We wrote tests before code, which looked backward. We released continuously, which looked dangerous.*

To many observers, this was a removal of constraints. The opposite was true.

*XP **compressed feedback loops** until truth became unavoidable. **Tests replaced promises**. **Continuous integration** replaced status reports. **Working software** replaced narrative. You could **no longer hide behind process** because the system itself reported your progress, loudly and continuously.*

*The practices that looked like chaos were actually mechanisms for enforcing honesty. **Pair programming** meant every line of code had a witness. **Test-first** meant you couldn't ship wishes. **Short iterations** meant you couldn't hide. The discipline was more demanding than what came before, not less. It just didn't look like the discipline people were used to seeing.*



Relocating Rigor - The Discipline That Looks Like Recklessness
Chad Fowler, 2026

***Extreme Programming displaced phase-gate development**. XP removed plans, design documents, and phase gates. These were the artifacts that made organizations feel safe. In their place it installed mechanisms that were far less forgiving: test-first development, continuous integration, constant peer review, real customer feedback.*

*It looked chaotic because **it removed the appearance of control**. What replaced it was **operational truth**. You knew where you stood because the code told you, not because a project manager updated a Gantt chart.*

***Continuous deployment displaced release management**. No release windows. No stabilization phases. No heroic integration efforts. Another apparent loss of discipline.*

*In reality, continuous deployment demands far stricter engineering than quarterly releases ever did. You need **reversibility**. **Observability**. **Automated verification**. **Fast rollback paths**.*



Relocating Rigor - The Discipline That Looks Like Recklessness
Chad Fowler, 2026

SCRUM

The Most Popular Agile Framework

THREE ROLES

Product Owner

Owns the WHAT — prioritizes value

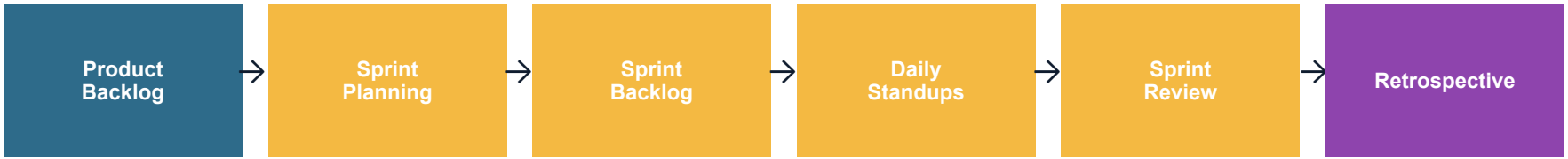
Scrum Master

Owns the HOW — facilitates process

Dev Team

Owns the EXECUTION — self-organizing

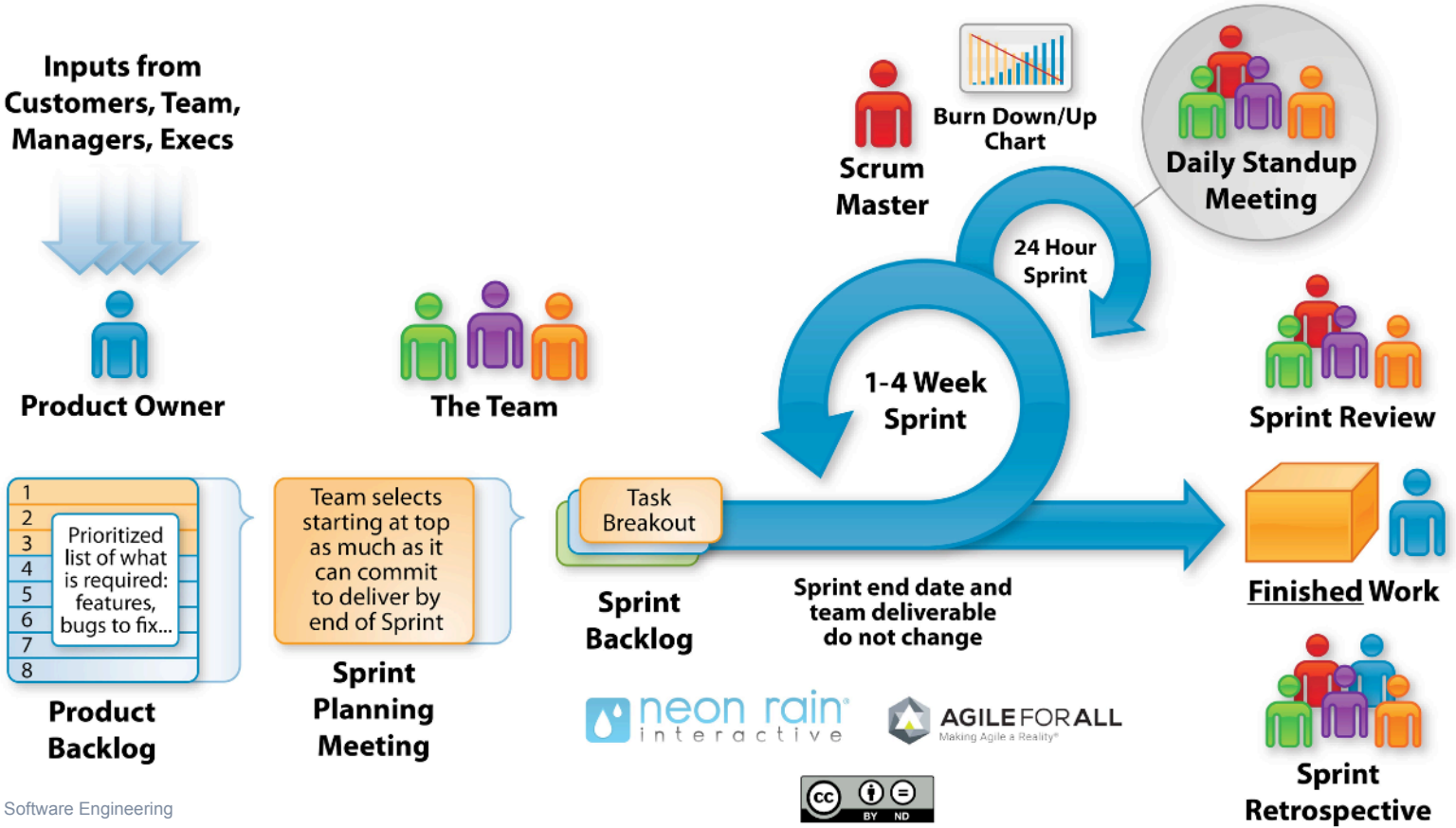
THE SPRINT (1-4 weeks, time-boxed)



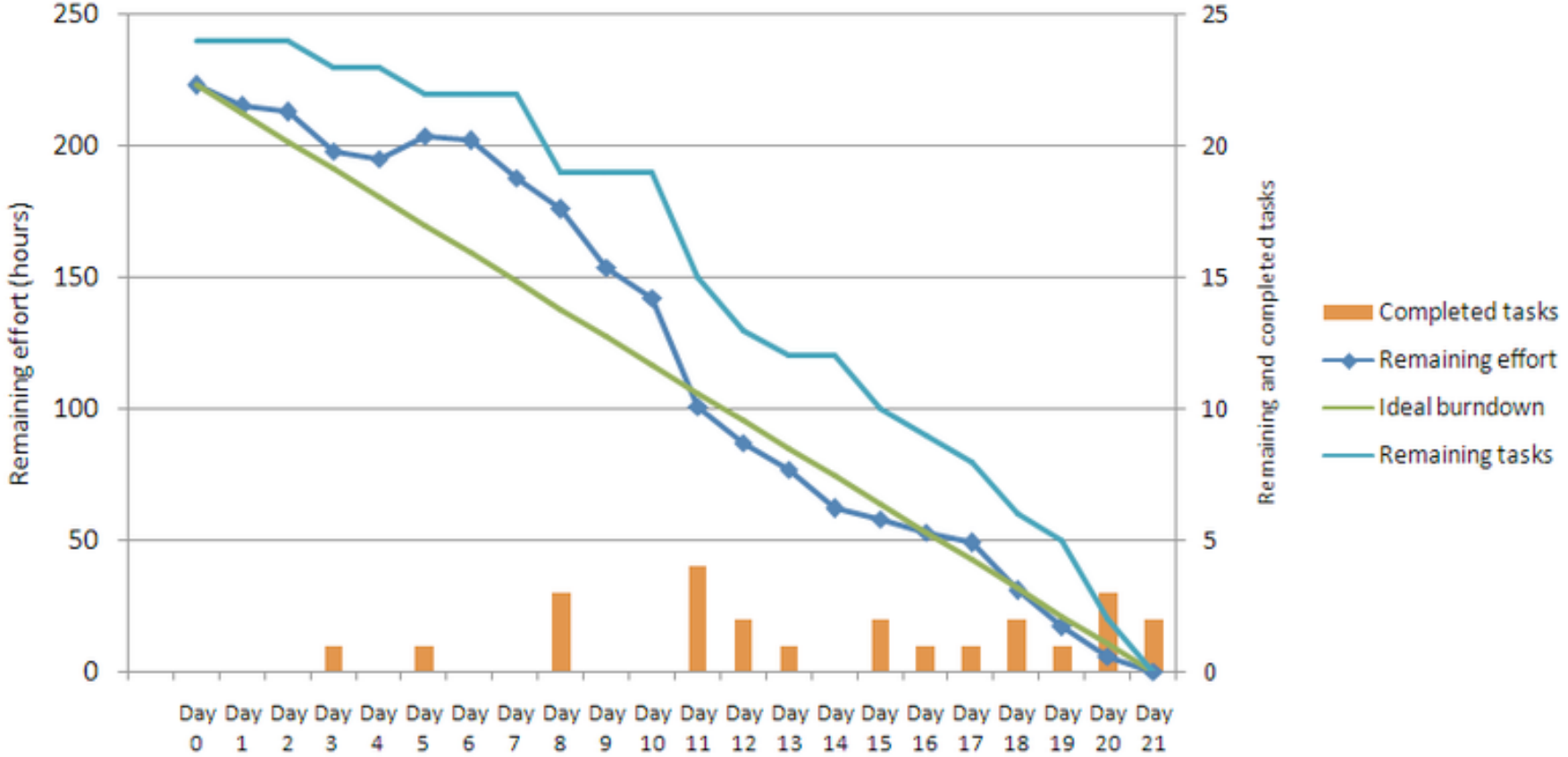
KEY ARTIFACTS

Product Backlog (prioritized wish list) → Sprint Backlog (this sprint's work) → Increment (shippable product)

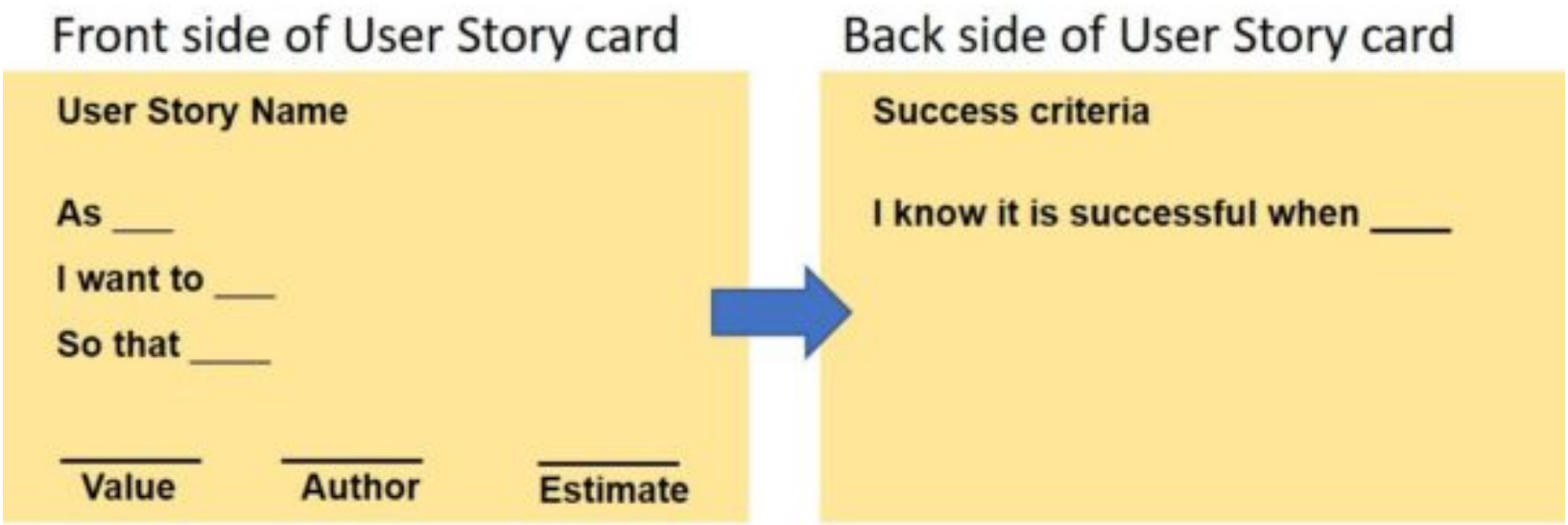
SCRUM


Inputs from Customers, Team, Managers, Execs
Product Owner
The Team
Scrum Master
Burn Down/Up Chart
Daily Standup Meeting
24 Hour Sprint
1-4 Week Sprint
Sprint Review
1
2
3
4
5
6
7
8
Prioritized list of what is required: features, bugs to fix...
Product Backlog
Team selects starting at top as much as it can commit to deliver by end of Sprint
Sprint Planning Meeting
Task Breakout
Sprint Backlog
Sprint end date and team deliverable do not change
Finished Work
neon rain interactive
AGILE FOR ALL
Making Agile a Reality®
Sprint Retrospective

BURNDOWN CHART

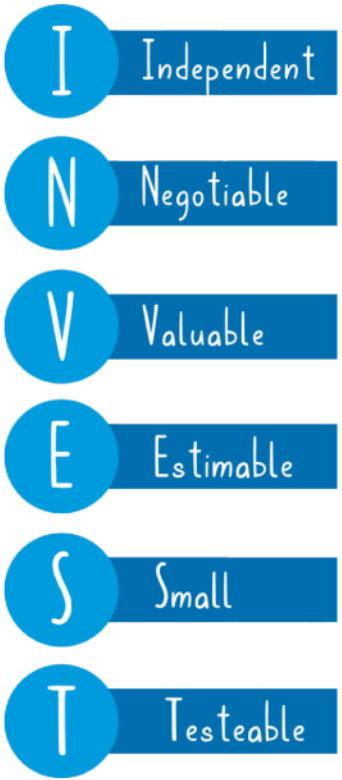


STORY CARDS



https://www.zentao.pm/blog/PO-How-to-write-user-stories-799.html

INVEST: Writing good User Stories



The story can be developed and delivered on its own

The story is a starting point for discussion, not a fixed contract

It delivers value to the end user or customer (not just technical work)

The team should be able to estimate the size or effort based on what's known

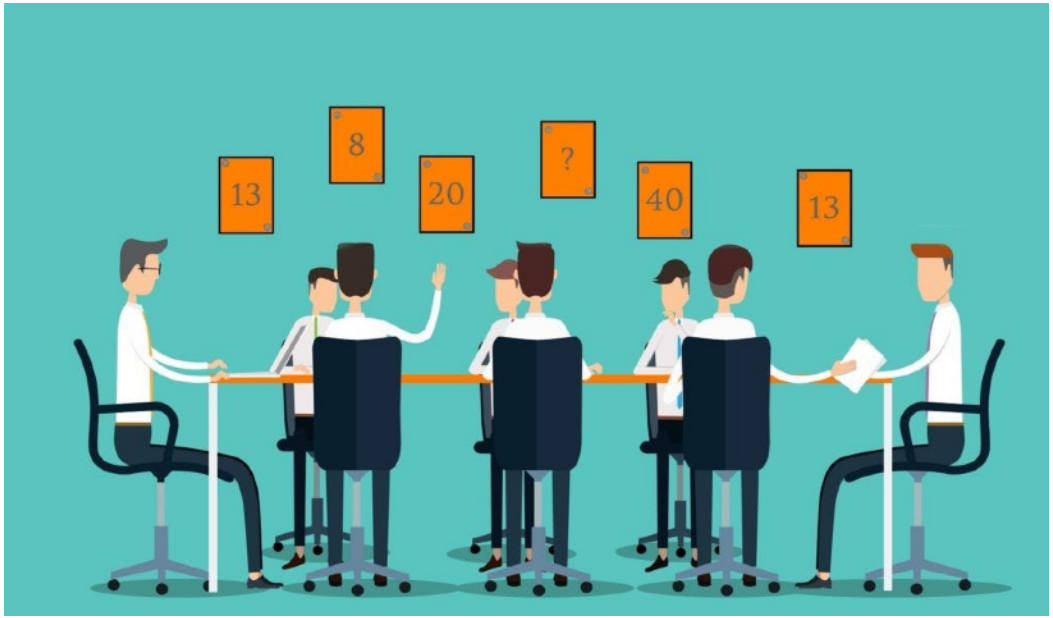
The story is small enough to be completed within a sprint or short time frame

The story must have clear acceptance criteria so it can be tested.

https://netmind.net/en/user-stories-and-the-acronyms-to-help-develop-good-ones-en/

PLANNING POKER

The idea is to help the team: to estimate together; uncover hidden complexity; and avoid one person dominate the estimate



PLANNING POKER - Example

How it works

- The Product Owner explains a story.
- Team members discuss it briefly and ask questions.
- Each person secretly picks a card with a number, often from a Fibonacci-like scale: **1, 2, 3, 5, 8, 13, 21**
- Everyone reveals their card at the same time.
- If estimates differ a lot, especially the highest and lowest explain why.
- The team discusses again and votes again until they get close enough to agree.

Story: "**As a** student, **I want to** reset my password **so that I** can access my account."

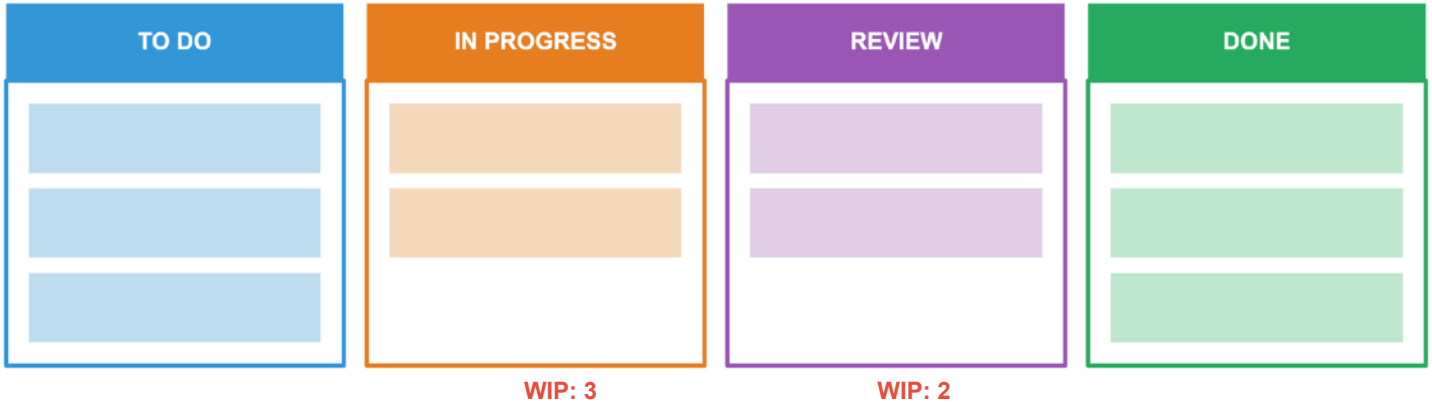
Votes:

- Alice: **3** ("*it is small because she assumed a basic reset flow*")
- Bob: **5**
- Carol: **13** ("*it is large because of email verification and security rules*")

After discussion, the team agrees on **8.**

KANBAN

Flow-Based Continuous Delivery. No iterations: release when ready. Focus on flow, not sprints.



KEY PRINCIPLES

Visualize work - Put all work on a board so everyone can see what exists, what is being worked on, and what is finished.

Limit WIP (Work In Progress) - Do not start too many things at once. Finish current work before pulling in more.

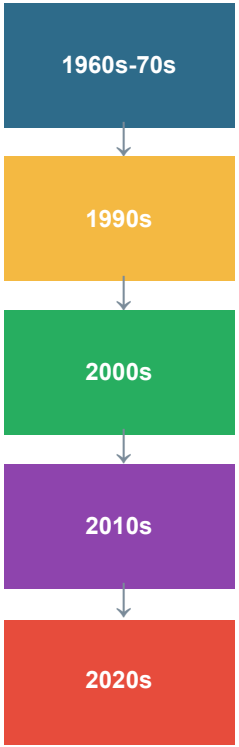
Manage flow - Watch how work moves through the system. If tasks get stuck, find the bottleneck and fix **it.**

Continuous delivery - You do not wait for the end of a Sprint: when something is done and ready, you can release it.

Kanban is great for a flow of continuous delivery,
but weaker at forcing regular planning, and review/reflection points.

THE EVOLUTION STORY

Processes evolve when constraints change. AI is the latest constraint shift. Adapt accordingly!



WATERFALL	Expensive compute, high risk
XP / AGILE	Internet speed, changing markets
SCRUM	Team coordination, enterprise adoption
KANBAN/LEAN	Continuous flow, DevOps
AI-NATIVE	Automation, abstraction

ENTER THE AI ERA

The Next Inflection Point in Software Development

> *"The rise of AI programming agents is changing software development **in the same way compilers did** in the time of Grace Hopper."*
>
> — Grady Booch

Another Level of Abstraction

Not a replacement for engineering a new tool that frees us from some tedious tasks

Amplifier, Not Magic

AI amplifies what you already have:
good practices → better results;
bad practices → faster "debt"

Systems Thinking Remains

Human judgment, responsibility, and understanding complexity still matter

AI agents do not remove software engineering.
They change what engineers spend their time on.

RELOCATING RIGOR

Chad Fowler's Framework for Understanding AI-Assisted Development

"Generative AI appears to remove the ultimate constraint: hand-written code. That makes people nervous, and it should. But the danger isn't probabilistic generation. The danger is quiet failure [...] The answer is to relocate the discipline. [...] Interfaces must be real contracts, not incidental boundaries. Evaluation must be ruthless. Failures must be loud and immediate.

The engineer's job shifts from typing code to specifying intent and verifying outcomes."

— Chad Fowler

THE PATTERN: "This has happened before!"

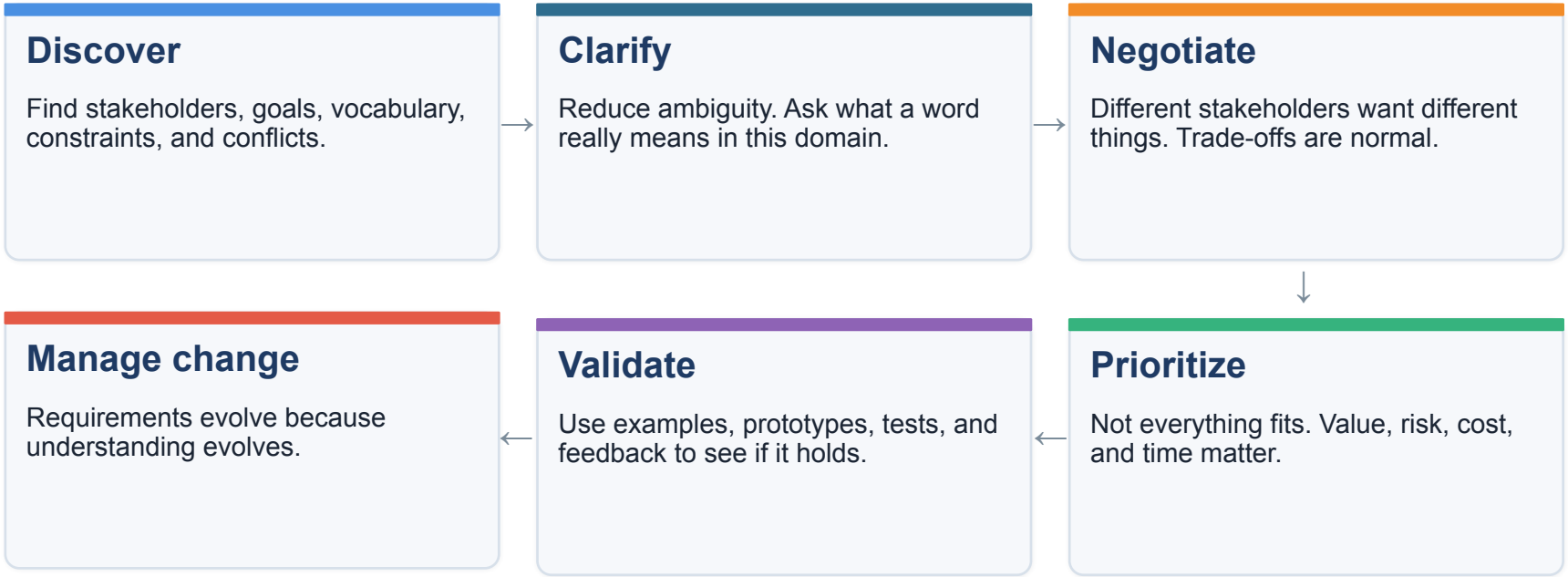
1990s: XP & Dynamic Languages	Upfront design documents → Tests + refactoring
2000s: Continuous Deployment	Release checklists → Automated pipelines + monitoring
2020s: AI-Assisted Coding	Line-by-line coding → Specs + tests + verification

REQUIREMENTS ENGINEERING

A requirement is not just “*what the system should do*".
It is also “*what the system must **not do***”,
“*what it may **assume***,” and
“*how we will know it **did the right thing***”.

WHAT REQUIREMENTS ENGINEERING ACTUALLY IS

*More than “writing requirements down”. It's a **continuous engineering** activity, not a bureaucratic document phase*



FOUR THINGS WE OFTEN CALL “REQUIREMENTS”

We tend to mix these up all the time

Functional behavior

What the system must do for actors.

Example:
Enroll a student in a course, if policy and capacity allow.

Quality attribute

How well it must do it.

Example:
Enrollment confirmation must appear within 2 seconds.

Constraint

A rule the solution must obey.

Example:
Must follow university policy & data-protection rules.

Assumption / risk

Something we currently believe, but may need to validate.

Example:
Most students will enroll from mobile devices.

WHY IS THIS MORE RELEVANT THAN EVER BEFORE?

Code generation got cheaper. Ambiguity got costlier.

1. Discover

Who are the **stakeholders**?

What **job** are we trying to help them do?

What **constraints** matter?

2. Specify

Turn vague wishes into **use cases**, **scenarios**, **acceptance criteria**, and **boundaries**.

3. Build + Verify

Only now does design, coding, testing, and AI-assisted implementation start to make sense.

4. Learn & Adapt

Real feedback changes our understanding, which means **requirements are never fully "done"**.

THE CORE SHIFT

What changed? What did not?

GROWS IN IMPORTANCE

- **Framing** the problem well
- Ask **sharper questions**
- Define **boundaries** & **constraints**
- Write **examples** & **acceptance criteria**
- **Verification: tests**, and **evals**
- **Critical judgment**

SHRINKS IN IMPORTANCE

- Memorize **diagram syntax**
- Hand-write **boilerplate specs**
- Treat documentation as **handoff ritual**
- Draw **diagrams by hand**

CHANGES SHAPE

- Modeling: lighter and faster
- Specs: inputs for **humans & agents**
- Analysis: **direct link to tests & evals**
- Human work: more **steering & review**

INTERMEZZO: A NEW ABSTRACTION LAYER

Martin Fowler on what LLMs change

> *"I think the appearance of LLMs will change software development to a similar degree as the change from assembler to the first high-level programming languages. [...] But this is more than a huge jump in abstraction [...] we're not just moving up the abstraction levels, we're moving sideways into non-determinism at the same time"*
>
> — Martin Fowler (2025)

New abstraction

You can ask for **larger chunks of behavior** instead of typing every step by hand.

But non-deterministic

Unlike classical compilers, LLMs can produce plausible but wrong outputs.
Requirements and checks matter more.

Implication for RE

Clear intent, examples, constraints, and **guardrails** become higher-leverage engineering work.

INTERMEZZO: A NEW ABSTRACTION LAYER

Martin Fowler on what LLMs change

... we're not just moving up the abstraction levels,
we're moving sideways into non-determinism at the same time "

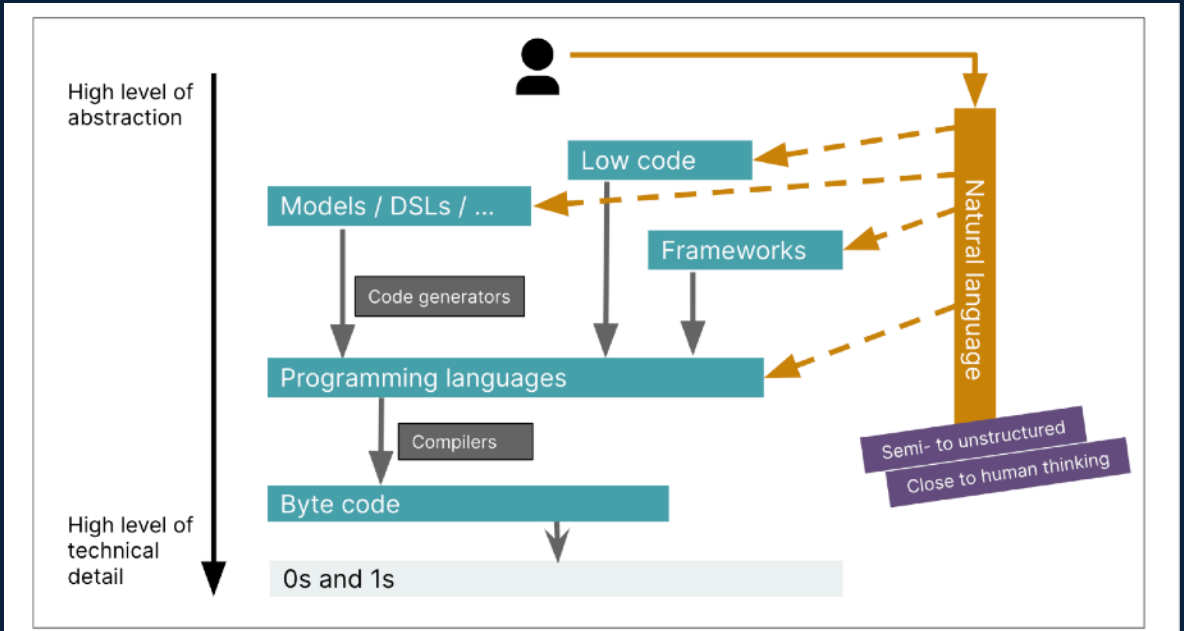


illustration: Birgitta Böckeler

FOUR THINGS WE OFTEN CALL “REQUIREMENTS”

We tend to mix these up all the time

Functional behavior

What the system must do for actors.

Example:
Enroll a student in a course, if policy and capacity allow.

Quality attribute

How well it must do it.

Example:
Enrollment confirmation must appear within 2 seconds.

Constraint

A rule the solution must obey.

Example:
Must follow university policy & data-protection rules.

Assumption / risk

Something we currently believe, but may need to validate.

Example:
Most students will enroll from mobile devices.

DISCOVERY STILL STARTS WITH PEOPLE

Classical requirements engineering got this right

Questions worth asking early

- Who **requested** this work?
- Who will **use** the solution?
- What **problem** hurts today?
- What would **“good”** look like?
- What must **never go wrong**?
- What **rules or policies** constrain us?
- What **assumptions** are we making?

AI can help, but cannot replace discovery

AI is good at:

- **transcribing** workshops
- **clustering** notes
- **summarizing** variants
- **drafting** first-pass artifacts

Humans must still own:

- **framing** the problem
- **deciding** trade-offs (incl. resolving **conflicts)**
- **confirming** what really matters

DISCOVERY WORKSHOPS

This enduring idea is to break the “us vs them” mindset.

BEFORE

Prepare just enough

Bring in...
short **problem statement**,
known **stakeholders**,
open **questions**, and
obvious **constraints**.

Do not arrive with a 40-page “finished” spec.

DURING

Work in shared language

Surface ...
vocabulary,
goals,
scenarios,
edge cases,
business rules, and
disagreements.

Whiteboard first; document second.

AFTER

Leave with concrete outputs

Use cases,
example **scenarios**,
key **quality attributes**,
open **issues**,
owners, and
the next experiment or **prototype**.

KEY TERMINOLOGY

USER STORY vs USE CASE vs SCENARIO vs ACCEPTANCE CRITERIA

User story

WHAT DO I WANT, AND WHY?

Capture user value
AS A / I WANT TO / SO THAT I

Best for:
prioritization & conversation.

Use case

HOW DOES THE SYSTEM SUPPORT THE USER'S GOAL?

Actor + goal + success path + boundaries.

Best for:
outside-in understanding.

Scenario

WHAT ARE CONCRETE WAYS IN WHICH THE USE CASE RUNS ?

Happy path, alternate path, or failure path.

Best for:
find edge cases & ambiguity.

Acceptance criteria

HOW DO WE CHECK THAT IT WORKS?

GIVEN / WHEN / THEN.

Best for:
verification/tests, and AI prompts that need precision.

EXAMPLE: Student Enrollment

USER STORY

Capture user value and start conversation. It is deliberately short / small / lightweight. It is not a full specification. It is mainly for prioritization and conversation

"This is a capability worth discussing and building"

AS A student,

I WANT TO enroll in a course

SO THAT I can build a semester schedule that fits well my goals

USE CASE

A Use Case takes the idea of a User Story and describes the goal-oriented functionality in more detail.

"Here is how the system helps the user achieve this goal”

Name: Enroll in Course

Primary actor: Student

Goal: The student successfully adds a course to their semester schedule.

Preconditions:

- The student is logged into the university system
- Enrollment period is open
- The course exists

Main success path:

1. The student searches for a course
2. The system shows the available course sections
3. The student selects a section
4. The system checks eligibility rules
5. The system checks seat availability
6. The system adds the course to the student’s schedule
7. The system confirms successful enrollment

Postcondition:

- The course appears in the student’s schedule
- The number of available seats is updated

Boundaries / notes:

- The system does not override prerequisite rules
- The system does not allow enrollment if registration is closed
- The system must prevent duplicate enrollment in the same course

SCENARIOS

one specific path through the Use Case. A Use Case has multiple Scenarios: happy path, alternate paths, failure paths

"Let's walk through some concrete example of what happens"

Scenario 1 — Happy Path

Student enrolls successfully in an available course

1. Student logs in
2. Student searches for "Software Engineering"
3. System displays available sections
4. Student selects Section A
5. System verifies prerequisites are satisfied
6. System verifies there is a free seat
7. System adds the course
8. System shows: "Enrollment successful"

Scenario 2 — Alternate Path

Selected section is full, but another section is available

1. Student searches for "Software Engineering"
2. Student selects Section A
3. System checks seat availability
4. System finds Section A is full
5. System displays: "Selected section is full"
6. System shows other available sections
7. Student selects Section B
8. System enrolls the student in Section B
9. System confirms success

Why this is an **alternate** path:

- the goal is still achieved
- but not through the main path

Scenario 3 — Failure Path

Student cannot enroll because prerequisites are missing

1. Student searches for "Advanced Algorithms"
2. Student selects a section
3. System checks prerequisites
4. System finds the student has not completed the required prior course
5. System blocks enrollment
6. System displays the reason for rejection

Why this is a **failure** path:

- the goal is **not** achieved

ACCEPTANCE CRITERIA

these are observable checks that tell us whether it works. Requirements become useful when they become checkable

"This is how we know that the system achieves the actor's goal"

Example checks for "Enroll in Course"

GIVEN a student who meets all prerequisites
WHEN the student enrolls in a course with free seats
THEN the system records an enrollment and shows confirmation

GIVEN a course with no free seats but an active waitlist
WHEN the student requests enrollment
THEN the system offers waitlisting

GIVEN a missing prerequisite
WHEN the student requests enrollment
THEN the system rejects the request and explains the specific rule

Kent Beck connection

With AI agents, tests and examples become even more valuable because they catch regressions the agent may introduce.

Examples are not "extra detail"; they are where ambiguity goes to die.

MINI EXERCISE: SPOT THE GAPS

What is missing from this requirement?

> *“The system lets students manage courses online.”*

Ask about the **actor**

Which student?
Any student?
Advisor acting for a student?
Anonymous visitor?

Ask about the **goal**

Enroll?
Drop?
View schedule?

Ask about the **rules**

Prerequisites?
Deadlines?
Capacity?
Payment?
Permissions?

Ask about success

What does a **correct outcome** look like?

USE CASES STILL MATTER

Outside-in thinking: actor + goal + value

A useful use case answers:
WHO wants WHAT value from the system?

Why keep them?

- force **user-centered** thinking
- define the **system boundary**
- **avoid premature design talk**

What they are not

- Not UI mockups
- Not class diagrams
- **Not implementation steps**
- Not a giant list of screens

AI "bonus"

A good use case is also a strong starting point for examples, tests, and AI-assisted implementation prompts.

ACTORS ARE ROLES, NOT JOB TITLES

The same person may play multiple roles

Good actor thinking

Actor = an external role that interacts with the system.

Examples:
- Student
- Academic Advisor
- Payment Service
- Notification Service
- Registrar Staff

Common mistake

“Professor”, “Admin”, or “User” are often too vague. Ask:

What distinct interaction does this role have?

One human, multiple roles

A staff member might act as:
- Registrar Approver
- Policy Maintainer
- Reporting User

If the interactions differ, the roles differ — even if the person is the same.

GOOD USE CASES LIVE AT SAME ABSTRACTION LEVEL

If one diagram mixes all three levels stakeholders stop trusting it

Kite Level

Business goal

Why the organization cares.

Example:
Manage student progression efficiently and fairly.

Sea Level

User-goal level

What an actor wants to accomplish with the system.

Example:
Enroll in a course.
Drop a course.
View current schedule.

Fish Level

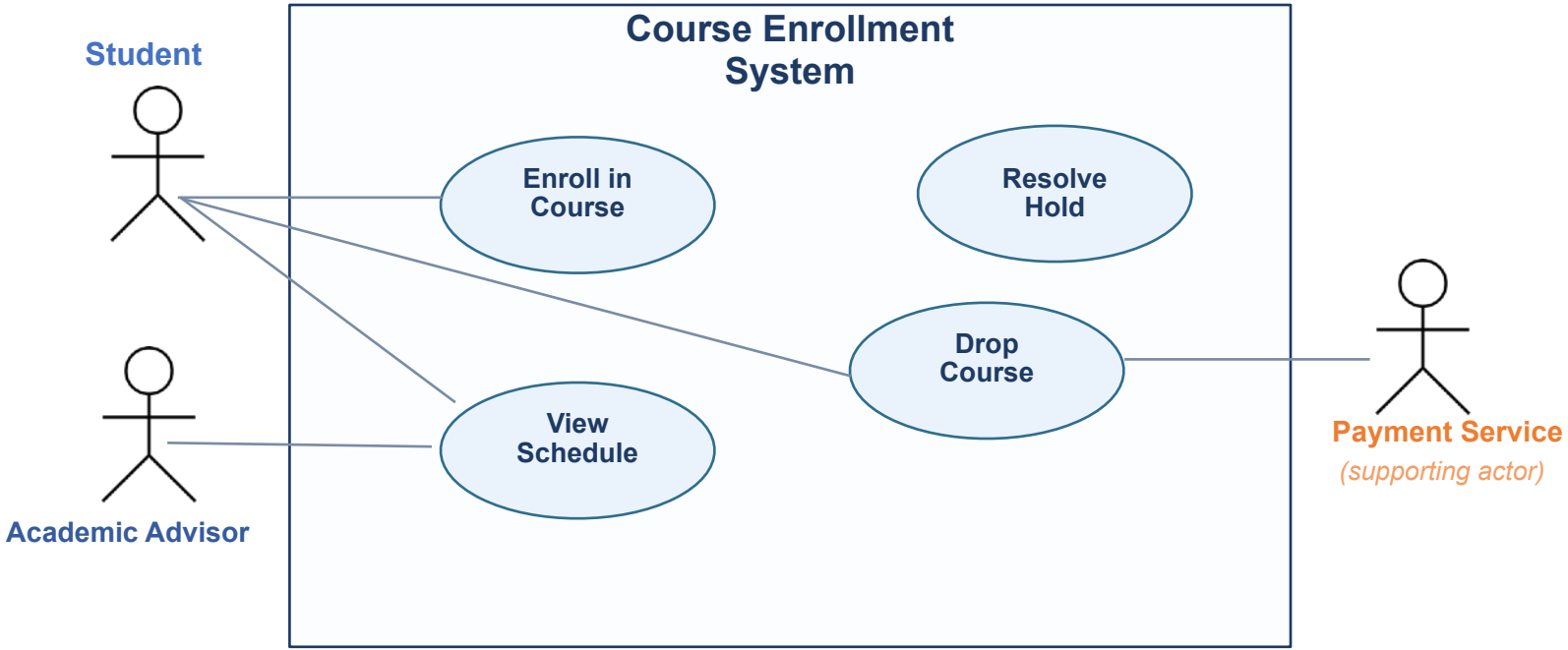
Sub-function level

Reusable smaller step.

Example:
Check prerequisites.
Calculate fee.
Send confirmation.

LIGHTWEIGHT USE-CASE DIAGRAM

Useful as a boundary picture — not as the whole specification



Only add <<include>> / <<extend>> when the extra arrow clarifies something. Most diagrams get worse when they overuse them.

QUALITY ATTRIBUTES ARE FIRST-CLASS REQUIREMENTS

Modern systems fail just as often on the “how well” as on the “what”

Reliability

What must never break?

Latency

How fast must key actions feel?

Security

Who must never access what?

Privacy

What data handling is acceptable?

Usability

Can users succeed without confusion?

Auditability

Can we reconstruct what happened?

Explainability

Can the system justify key decisions?

Cost

What usage or inference budget is acceptable?

REQUIREMENTS vs. CONSTRAINTS vs. ASSUMPTIONS

Type	Meaning	Example	What you do next
Requirement	Desired system behavior or quality	“Student can enroll in an offered course.”	Model it, refine it, test it.
Constraint	A boundary the solution must obey	“Must comply with policy and data rules.”	Surface it early; design around it.
Assumption	Something believed but not yet proven	“Most students enroll from phones.”	Treat it as a risk; validate it.

Do not write all three as if they were the same thing!



Conference

Codecamp in Timisoara 2026

21 May 2026

Timisoara - Iulius Congress Hall 99 euro

register



Codecamp Day

A full day of pure technical insight. Parallel tracks with no fluff – just code, architecture, and real-world lessons from the trenches.

	THE BLUEPRINT	ENGINE ROOM
10:00 - 10:45	**Making Architecture Alive Again: Towards a practice of building theories of living systems, James Coplien**	
11:00 - 11:45	**How to Teach Your Agents about architecture, Neal Ford**	
12:00 - 12:45	AI & Solution Architecture: How AI is Transforming the Way We Design Solutions, Catalin Gheorghiu	The Rise of the Agent-Driven Engineer, Mihai Anton
12:45 - 14:00	Lunch Break	
14:00 - 14:45	Death by Tool Calls: Using CodeGen Agents to Solve the Biggest Problem with Agentic AI, Jeff Prosise	Why software breaks, and how we can fix it, Jules May
15:00 - 15:45	When Agents Code, Where Does Understanding Go?, Radu Marinescu	System Drift and how to live with it, James Lewis
16:00 - 16:45	**Machines, Learning, and Machine Learning, Dylan Beattie**	

REQUIREMENTS CHANGE BECAUSE SOFTWARE LIVES

The old myth

"*If we work hard enough, we can get the requirements fully right once, upfront, and freeze them.*"

The better view

Many requirements start as **hypotheses**.

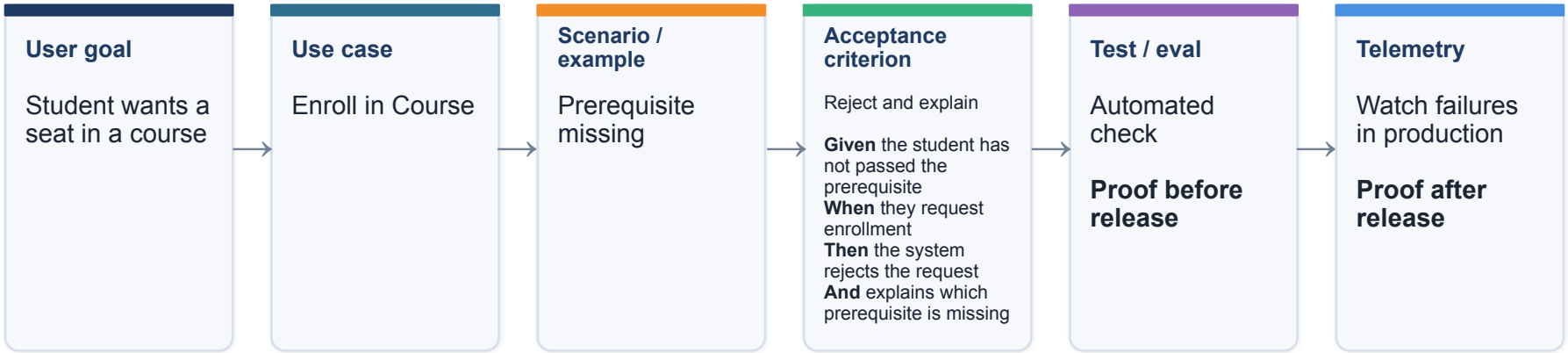
We learn by building, testing, observing, and revising.

Practical loop

Hypothesis
↓
Scenario / example
↓
Prototype or implementation
↓
Feedback / telemetry
↓
Refined requirement

TRACEABILITY AS A LIVING CHAIN

A lightweight chain from intent to evidence



It becomes a living chain only if we keep the links current: when use cases change, scenarios, criteria, tests, and telemetry must change too.

This does not happen automatically!
It's an engineering discipline of keeping intent connected to checks and real-world evidence over time.

INTERMEZZO: RELOCATING RIGOR

Chad Fowler's lens for AI-assisted development

> *"Constraint removal is mistaken for loss of rigor. But what actually happens is rigor relocation. Control does not disappear; it moves closer to reality."*
>
> **— Chad Fowler (2025)**

Old focus

Rigor looked like manually typing code, large upfront docs, and ceremonial review.

Where it moves

Toward **explicit specs**, contracts, tests, evaluations, monitoring, and guardrails.

Implication for RE

Writing intent precisely and **making success observable** becomes more important.

INTERMEZZO: SPEC-DRIVEN DEVELOPMENT

One of the most discussed AI-assisted engineering ideas of 2025–2026



> *"Spec-driven development uses well-crafted software requirement specifications as prompts, aided by AI coding agents, to generate executable code."*
>
> **— Thoughtworks / Liu Shangqi (2025)**

Why people care

It promises better leverage from AI than vague vibe-coding prompts.

Why it matters here

Requirements quality directly shapes code quality when agents consume the spec.

REMEMBER!

Spec-driven work is powerful — but only with short loops, checks, and human review.

INTERMEZZO: SPEC-DRIVEN DEVELOPMENT



Use well-crafted specs as structured prompts for AI coding agents, not vague “build this” instructions.

More than a PRD

- external behavior
- inputs / outputs
- business rules
- constraints

Not “code disappears”

- specs drive generation
- code still must be reviewed
- tests keep truth executable

Why now?

- agents have larger context
- natural language became executable-ish
- bad prompts scale bad code

SPECS IN THE AI ERA

From documents for humans to shared instructions for humans + machines

Classical good spec

- clear scope
- stable vocabulary
- major scenarios
- business rules
- constraints

Still relevant? Absolutely.

Now add

- **pre/postconditions**
- **invariants**
- **interface contracts**
- **example inputs/outputs**
- **unknowns** and **risks**

Why this matters more

AI agents consume specs.

If the spec is vague, the agent fills the gaps with guesses.

That makes high-quality specs a multiplier.

WHAT A GOOD SPEC GIVES THE AGENT

The spec is engineered context: behavior, rules, and boundaries

Behavior

- Given/When/Then scenarios
- example inputs/outputs
- success + failure paths

Conditions

- preconditions
- postconditions
- what must be true before/after

Invariants

- rules that must never break
- e.g., no duplicate enrollment

Contracts

- interface types
- integration contracts
- clear inputs and outputs

Risks

- unknowns
- constraints
- where agent must not guess

Domain language, concrete examples, clear structure, concise coverage of the critical path remain essential!

IS THIS JUST WATERFALL AGAIN?

With AI agents, teams may be tempted to write complete specs upfront and hand them to the machine. This can push teams toward Waterfall-like batch patterns, losing the benefits of iterative feedback!

❌ THE ANTI-PATTERN

- Write exhaustive spec upfront
- Hand entire spec to AI agent
- Get massive code dump
- Struggle to verify & integrate
- Late discovery of problems

✓ THE PATTERN THAT WORKS

- Small, focused specs
- Human review
- TDD: tests **before** AI generates code
- Agent implements
- CI / Tests / Evals
- Refine spec + code
- Iterate rapidly

Guardrails

- **Human-in-the-loop:** spec review is iterative
- **Architecture constraints:** rules / AGENTS.md / tool context
- **Deterministic CI/CD:** quality checks still protect the system
- **Watch drift:** spec, code, and tests must stay aligned

AI-SPECIFIC REQUIREMENTS FOR AGENTIC SYSTEMS

Agentic systems need requirements about permissions, approvals, fallback, logging, and uncertainty handling — not just features

Permissions

What tools may it call?

Approvals

Which actions require human confirmation?

Data boundary

What student data may it see?

Fallback

When must it hand off to a human?

Audit trail

What decisions and tool calls must be logged?

Uncertainty

How should it behave when confidence is low?

Safety / policy

What must it never do?

Cost / rate limits

How much autonomous work is acceptable?

MINI EXERCISE: REWRITE A DANGEROUS AI REQUIREMENT

From vague automation to bounded behavior

Weak requirement

"The agent should help students with enrollment automatically."

What is missing?

- which **actions**?
- how much **autonomy**?
- what **approvals**?
- what **policy boundaries**?
- what happens on **uncertainty**?

Stronger rewrite

"The AI Enrollment Assistant **may recommend course selections and prepare an enrollment request**,
but

- it must ask for explicit confirmation before submitting or paying fees;
- it must cite the policy checks it used,
- log its tool calls, and
- hand off to a human advisor if prerequisites rules conflict."



SOFTWARE ANALYSIS: MODELLING REQUIREMENTS

FROM “WHAT” TO “WHO”

Why analysis models exist



Use cases

Group the major goals and define the system boundary.

Scenarios

Turn one goal into concrete paths and expose ambiguity.

Responsibilities

Ask which objects or services should know, do, and collaborate.

OBJECT-ORIENTED ANALYSIS, REFRAMED

Still useful, but not the only destination

OOA helps when...

- the domain has stable concepts
- responsibilities matter
- data and behavior should stay together
- students need to learn decomposition

Ask these questions

- What are the important objects?
- What should each object know?
- What should each object do?
- Who should collaborate with whom?

But...

... **not every system ends naturally as a class diagram!**

OOA is useful, but not every system ends naturally as a class diagram!
Services, workflows, events, and agents also matter.

CRC CARDS: AN OLD TOOL TEACHING THE RIGHT HABIT

Class • Responsibility • Collaborator

Why still relevant?

Because it forces behavior-first thinking:

- Who owns this responsibility?
- Who must collaborate?

Class	
Responsibilities	**Collaborators**

Modern update

AI can generate candidate CRC decompositions quickly.

The human skill is now to critique, merge, prune, and test them.

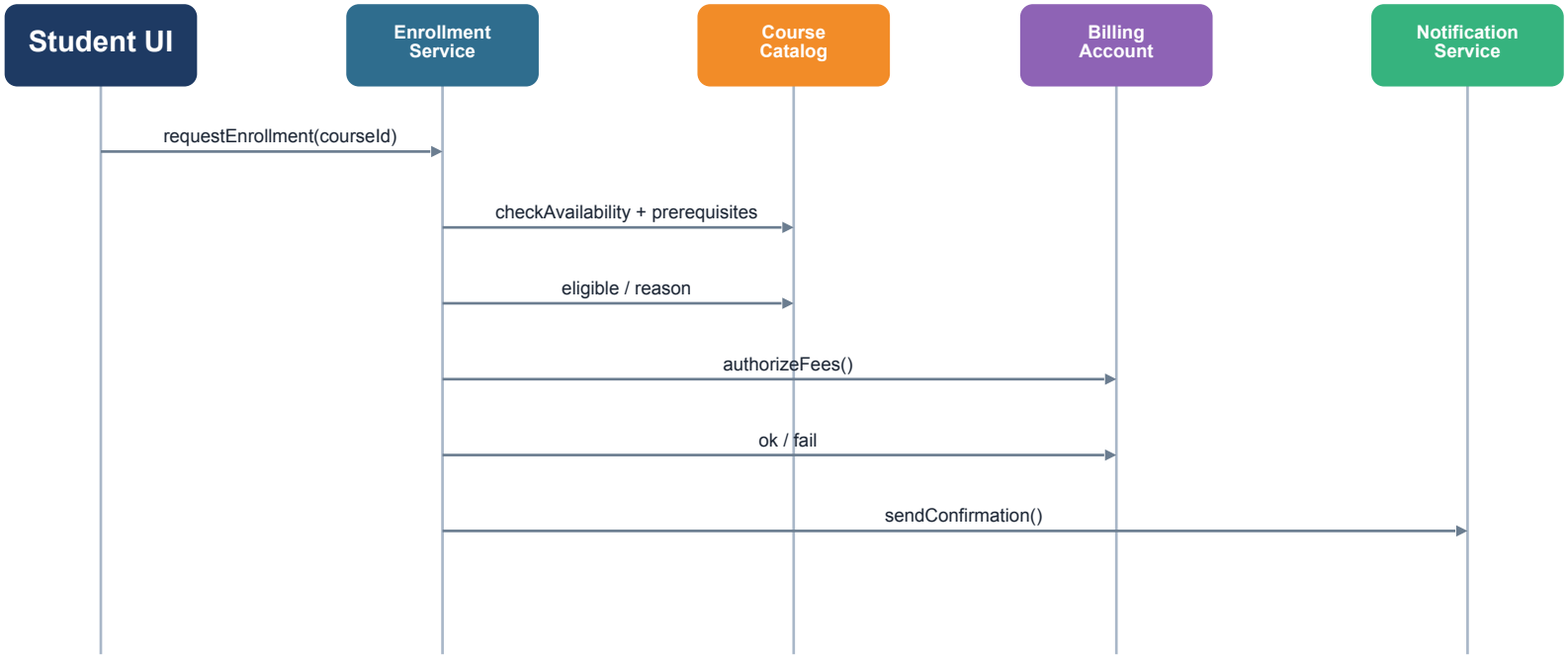
HEURISTICS FOR GOOD RESPONSIBILITIES

Rebecca Wirfs-Brock - "Responsibility-Driven Design"

1. **Keep behavior near the data it needs**
2. **Give each class or service one clear purpose**
3. **Collaborate instead of centralizing everything**
4. **Name responsibilities as verbs, not vague labels**
5. **Prefer meaningful behavior over getters/setters everywhere**

SEQUENCE DIAGRAMS: STORIE BECOME INTERACTIONS

Use them when a scenario crosses multiple objects or services



A sequence diagram tells the story of one scenario.
It is not the whole domain model!

CLASS DIAGRAMS

A class diagram describes possible objects and links, not one running system snapshot



Magritte reminds us: a picture is a representation, not the thing itself.

Class

A type / template

Example:
Student
CourseOffering
Enrollment

Object / instance

A concrete thing at runtime

Examples:
radu:Student
se2026:CourseOffering
e17:Enrollment

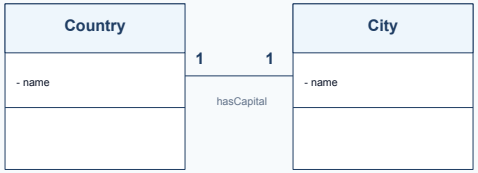
Class diagram details constrain how object instances can be connected.

A class diagram is a model of what kinds of objects may exist and how they may be linked.

MULTIPLICITY TALKS ABOUT OBJECTS

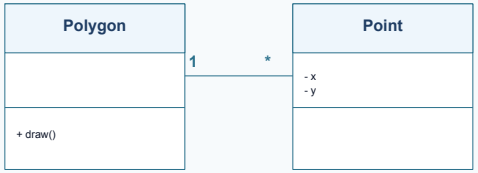
The numbers on an association end constrain how many instances may be linked

1-to-1 association



Object meaning:
Each Country object has exactly one City object as capital. Each City object is capital of exactly one Country object in this model.

1-to-many association



Object meaning:
One Polygon object is linked to many Point objects. The points are the runtime vertices of that particular polygon.

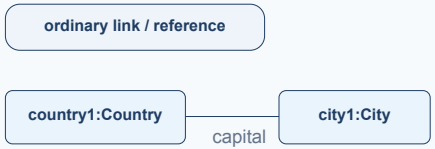
Common mistake:
reading multiplicity as if it counts classes.
It does not! It counts object links at runtime.

ASSOCIATION, AGGREGATION, COMPOSITION

The difference is what happens to the object links and lifetimes

All three are relationships between classes, but their meaning is about object instances.

Association

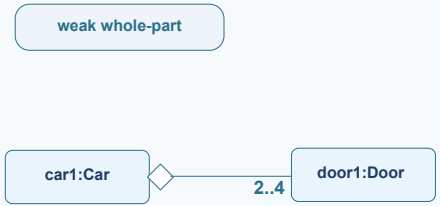


Object meaning:

Two objects know or use each other. Their lifetimes are independent. The link can be created, changed, or removed without destroying either object.

Example: Country -- City

Aggregation

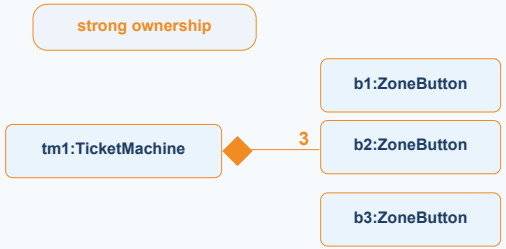


Object meaning:

There is a whole-part link, but parts may be replaceable or may continue to exist outside the whole. The diamond marks the whole side.

Example: Car has Engine and Doors

Composition



Object meaning:

The part belongs to exactly one whole in this model. If the whole is deleted, the parts are deleted or become meaningless too.

Example: TicketMachine owns ZoneButtons

Class diagrams describe object-level rules:

who may reference whom, how many links are allowed, and whether parts survive independently.

COMMON MODELING TRAPS

Behavior first, not noun collecting!

Trap 1: every noun becomes a class

Read a paragraph, circle nouns, and create a bloated object model.

Trap 2: external actor inside the model

A user of the system is not a domain class inside the system.

Trap 3: inheritance too early

General taxonomies before they understand behavior and variation might be misleading

Roles vs classes

Different behavior may justify different classes. Different labels alone do not.

Example: “Mother” and “Father” might both just be Person objects playing roles.

LLMs are very good at generating too many plausible classes.
That makes critique and pruning even more important.

WHAT AI CHANGES IN MODELING WORK

Generation gets cheaper; judgment gets more valuable

CHEAPER

- drawing first-pass diagrams
- generating alternative decompositions
- drafting example scenarios

MORE VALUABLE

- spotting **missing constraints**
- defining **invariants** and **boundaries**
- checking what success really means

MORE DANGEROUS

- “looks right” replacing actual verification
- false feeling of completeness
- hallucinated classes and rules
- hidden assumptions

SKILLS FOR TH FUTURE SOFTWARE ENGINEERS

What grows, what shrinks, what changes

GROWS

- asking sharper questions
- writing precise examples
- defining boundaries and constraints
- reasoning about trade-offs
- verification, tests, evals
- critical review of AI output

SHRINKS

- memorizing notation tables
- drawing boilerplate diagrams by hand
- producing documentation no one uses
- mistaking artifact quantity for rigor

CHANGES

- specs become machine-consumable
- modeling becomes faster and lighter
- coding becomes more supervisory
- documentation becomes living context

INTERMEZZO: HUMANS STEER. AGENTS EXECUTE.

Why “on the loop” beats both micromanagement and vibe coding

Humans outside the loop

Too little review.

Too much trust.

Fast, exciting, fragile.

This is where vibe coding breaks down on serious systems.

Humans inside the loop

Too much hand-holding.

Every tiny step needs approval.

Safe, but the human becomes the bottleneck.

Humans on the loop

Humans set goals, boundaries, tests, and guardrails.

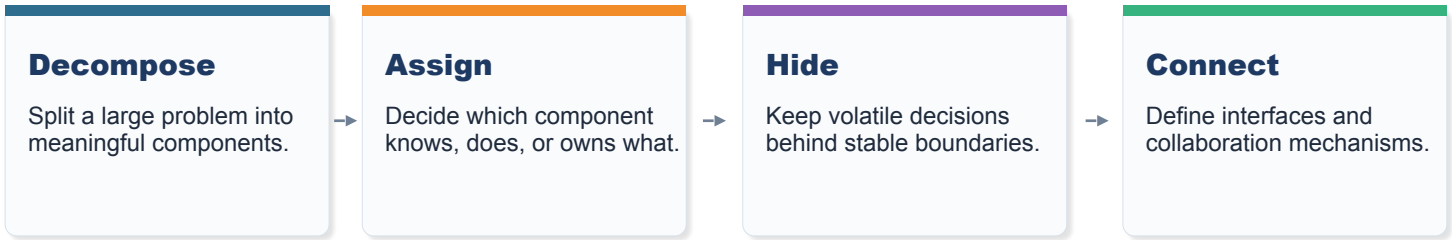
Agents execute within that harness.

This is the sweet spot.

SOFTWARE DESIGN & ARCHITECTURE

THE ROLE OF SOFTWARE DESIGN

*Design is about making the system **understandable**, and **changeable**, **buildable***



Design is not “drawing boxes”!
Design is deciding what the boxes mean, why they exist, and how they can change independently.

WHEN DO WE NEED DESIGN?

Small script

One person understands everything. A simple mental model may be enough.

Design matters, but consequences are local.

Growing system

More features, more modules, more data, more change.

Bad structure slows every future change.

Real product

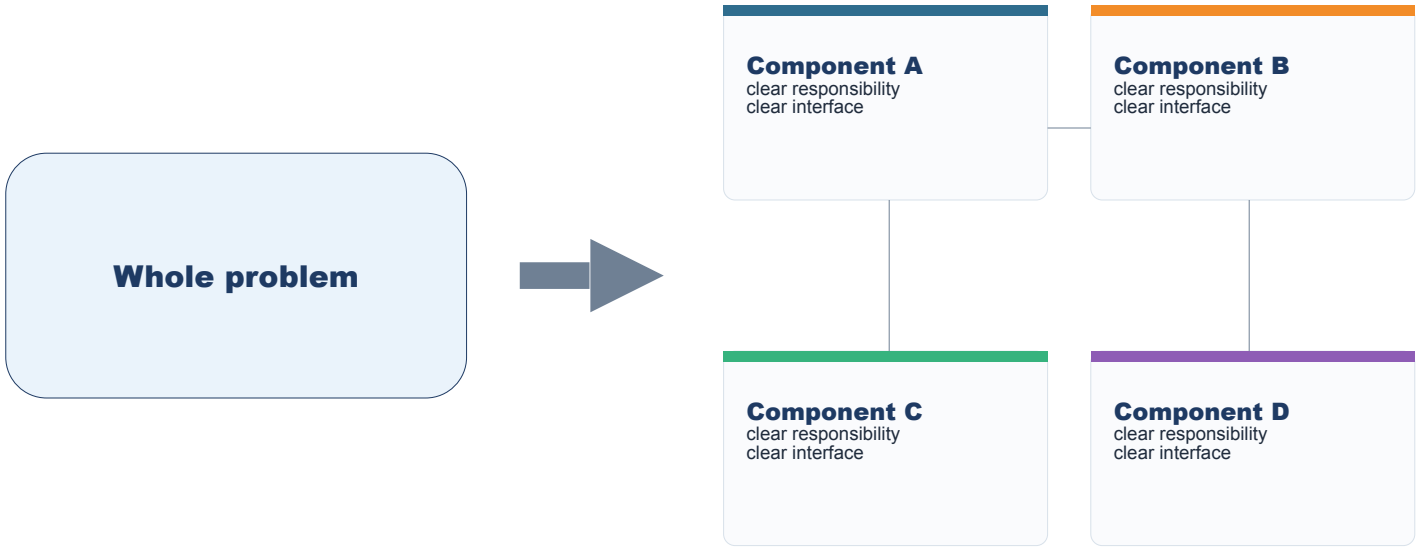
Multiple teams, integrations, users, releases, operations, and years of evolution.

Design becomes a survival mechanism.

Design becomes necessary when complexity exceeds what one mind can hold

DECOMPOSITION: DIVIDE AND CONQUER

But the hard part is choosing the right cuts



Good decomposition creates pieces that can be understood, built, tested, and changed with minimal surprise.

A COMPONENT IS MORE THAN A FILE

It is a boundary around an abstraction and a design decision

Component

A software entity that encapsulates an abstraction.

It has a name, a boundary, and a responsibility.

Interface

What other components are allowed to depend on.

This is the public contract.

Hidden decision

Internal details that may change without forcing everybody else to change.

A component is useful when it reduces what other parts of the system need to know.



WHAT IS “GOOD DESIGN”?

*It's not a precise answer, like in math! It's a **trade-off** that proves itself over time*

The tempting view

“There must be one correct design.”

“Follow a method and it will guarantee the right answer.”

This feels comforting, but real systems rarely behave like textbook exercises.

The engineering view

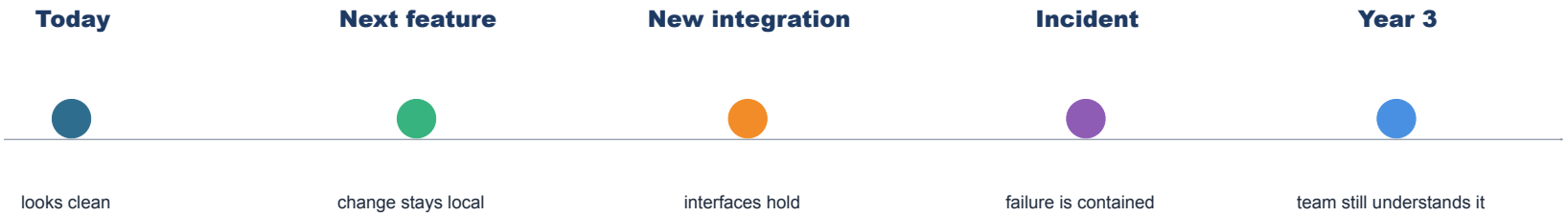
Good design balances **trade-offs** and reduces total **lifetime cost**.

It is judged by how it handles change, defects, integration, and future understanding.

There is no perfect design. There are decisions with consequences.

GOOD DESIGN IS REVEALED BY CHANGE

A design decision is tested by the next requirement, not by the diagram



Design quality is temporal:

a choice that is “fine today” may be expensive tomorrow.
That is why design is about future cost of change.

MODULARITY: MEYER’S VISION

Autonomous elements connected by a coherent, simple structure

> *“A software construction method is modular if it helps designers produce software systems made of autonomous elements connected by a coherent, simple structure.”*
>
> **— Bertrand Meyer**

Autonomous

A module can be understood and changed without dragging the whole system into your head.

Connected

Modules still collaborate through explicit, well-defined interfaces.

Coherent

The overall structure makes sense; it is not just a pile of isolated parts.

MEYER’S FIVE CRITERIA

Five ways to ask whether a design is genuinely modular

Decomposability

Can we split the system into manageable parts?

Composability

Can we reuse parts to build new systems?

Understandability

Can one part be understood on its own?

Continuity

Do small changes stay local?

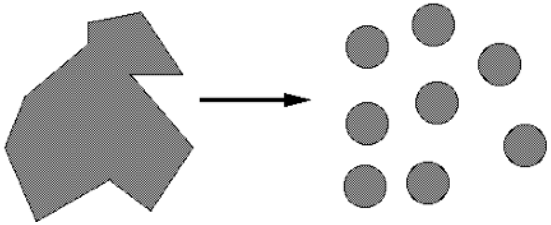
Protection

Do failures stay contained?

A modular design is not just “many classes”.
It is a structure that supports change, reuse, and fault isolation.

1. DECOMPOSABILITY

Can the problem be split so different people can work independently?



Good example

Course Enrollment System →
- Course Catalog
- Enrollment Service
- Billing Adapter
- Notification Service

Each part has a clear purpose and limited dependencies.

Counter-example

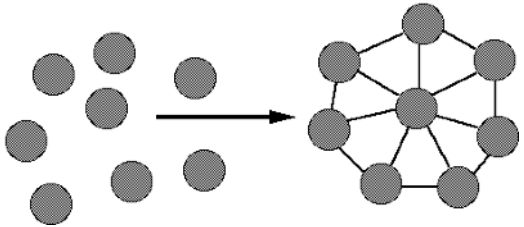
“InitializationManager” that sets up everything for everybody.

It knows too much, touches too much, and becomes a hidden central dependency.

Key Question:
Can two teams implement these pieces without constantly blocking each other?
If not, the split is probably poor.

2. COMPOSABILITY

Can parts be combined in new ways without rewriting them?



Good example

UNIX-style tools: one component filters, another sorts, another counts.

Each tool does one thing and can be recombined in many pipelines.

Software example

Payment adapter can be reused by enrollment, library fines, and dorm fees because it exposes a clean contract.

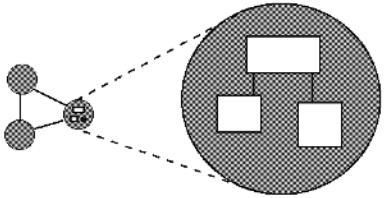
Bad sign

A module only works if the whole application is present.

It is technically “separate” but not reusable.

3. UNDERSTANDABILITY

Can a human understand one module without reading the whole system?



Good module

The module exposes a small concept:

“EnrollmentService coordinates enrollment decisions.”

You can read its interface, examples, and tests and understand its role.

Bad module

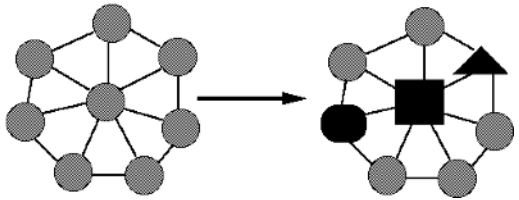
The module makes sense only after reading five other modules in a precise order.

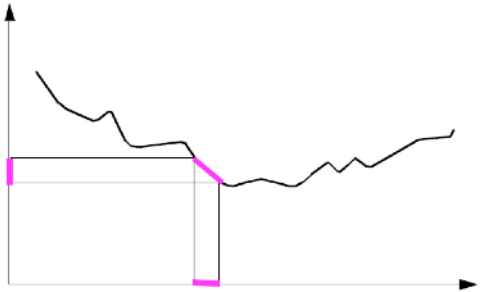
It has hidden sequential dependencies and context you must “just know”.

A module that is small but mysterious is not understandable.
A module that is larger but conceptually clear may be better.

4. CONTINUITY

When requirements change, does the impact stay local?





Good example

Changing maximum credits from 30 to 32 changes one policy object or configuration value.

The rest of the system keeps working.

Counter-example

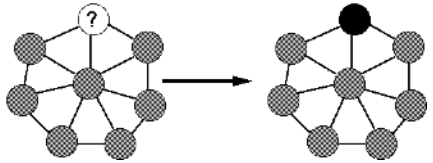
The credit limit is copied into UI, API, database scripts, tests, and reports.

One small policy change causes scattered edits.

Design question:
Where should this rule live so that future changes touch the fewest places?

5. PROTECTION

When something goes wrong, is the damage contained?



Good example

Validate input at the boundary.

If a course ID is invalid, reject it before it reaches enrollment rules, billing, and notifications.

Counter-example

Undisciplined exceptions and unchecked nulls propagate through the system.

One invalid input causes unrelated modules to fail.

In the Agentic Coding era protection also means guardrails: permissions, evals, rollback paths

MEYER'S FIVE MODULARITY RULES

Direct Mapping	Solution structure should reflect the problem structure.
Few Interfaces	Each component should talk to few others.
Small Interfaces	When components communicate, exchange little information.
Explicit Interfaces	Communication must be visible, not hidden.
Information Hiding	Hide volatile design decisions behind stable interfaces.

The rules are simple to state...
but they require discipline every time we introduce a dependency.

RULE 1: DIRECT MAPPING

The solution should resemble the problem enough that change is predictable

Good mapping

Domain idea:
Student enrolls in Course Offering.

Design idea:
EnrollmentService coordinates *Enrollment* objects for *CourseOfferings*.

A policy change has an obvious home.

Bad mapping

Everything is routed through generic modules like Processor, Manager, Handler, Utility.

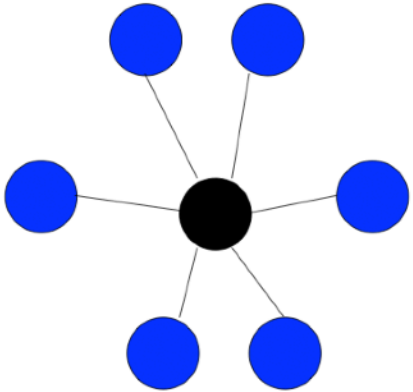
The code structure hides the domain structure. Every change becomes a search expedition.

Direct mapping helps continuity:
when the world changes, you can guess where the software must change.

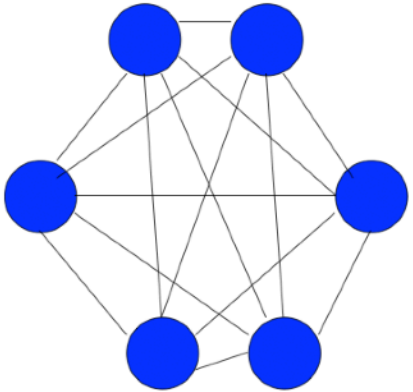
RULE 2: FEW INTERFACES

Don’t talk to many components if you can avoid it

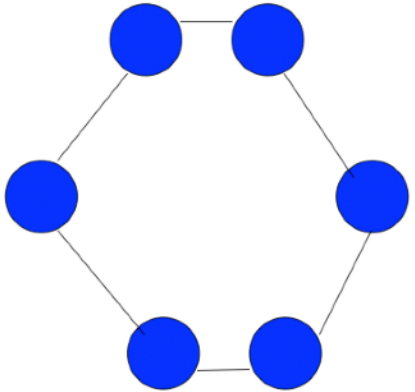
Centralized



Anarchic



Distribute



Anarchic dependency graphs are where changes disappear into meetings, bugs, and “why did this break?” moments.

RULES 3 AND 4: SMALL + EXPLICIT INTERFACES

Don’t talk a lot. Don’t whisper.

Small interface

If EnrollmentService talks to Billing, it should send only what Billing needs:

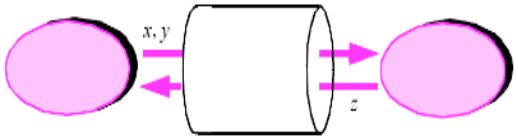
studentId, amount, purpose

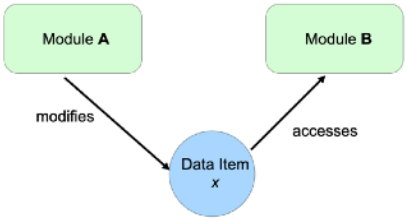
Not the entire Student object, full schedule, and internal state.

Explicit interface

If A depends on B, the dependency must be visible in code, config, or interface definitions.

Hidden shared data and global state create invisible coupling.

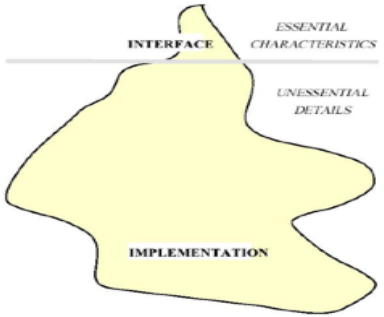




few interfaces = “don’t talk to many”;
small interfaces = “don’t talk a lot”;
explicit interfaces = “talk loud and in public.”

RULE 5: INFORMATION HIDING

Hide decisions that are likely to change behind stable interfaces



Client code

Depends on the interface, not the hidden details.

Public interface

Hidden details

- data structure
- algorithm
- database schema
- caching strategy
- API provider

Information hiding is change control:
the less the outside world knows, the more freely the inside can improve.

INTERMEZZO: CHEAP CODE MAKES DESIGN MATTER MORE

Agentic coding does not change the need for structure

> *“When code becomes cheaper, the scarce resource becomes understanding: what to build, what not to build, and how the pieces fit together.”*
>
> **— Kent Beck, theme from 2025 interviews**

Less scarce

Typing boilerplate, generating variants, drafting APIs, producing first-pass tests.

More scarce

Coherence, boundaries, invariant thinking, trade-off judgment, system-level ownership.

Design lesson

If generation is fast, bad structure spreads faster too. Design is the governor.

INTERMEZZO: SYSTEM-LEVEL DESIGN IS BACK

John Ousterhout's old warning becomes sharper in the AI era

Tactical programming

"Make this task work now."

AI agents are very good at producing plausible local solutions quickly.

Risk: shallow fixes, scattered complexity, accidental coupling.

Strategic design

"Make future work easier."

Think about modules, interfaces, deep abstractions, and long-term simplicity.

Risk if ignored: faster delivery today, slower everything tomorrow.

Design is the discipline of paying a little now to avoid paying forever.

INTERMEZZO: WHAT AGENTS ARE GOOD AT TODAY

They are powerful assistants, but not architects on "auto-pilot"

Local transformations

Agents are strong at localized edits, refactorings, scaffolding, and consistency fixes.

This is useful — but local improvement is not the same as system design.

System coherence

High-level design asks: where should responsibilities live? What boundaries survive change? What failures must be contained?

Human responsibility

The engineer must review generated structure, not just generated syntax. “Looks plausible” is not a design criterion.



FROM DESIGN TO ARCHITECTURE

Architecture starts where decisions become hard to change

Design decisions — **Some decisions become structural constraints** — **Architecture**

Easy to change later

Button labels, local algorithm choices, small function bodies.

Harder to change

Data model, service boundaries, communication style, deployment topology.

Architecture-level

Decisions many components depend on — the ones that shape future options.

WHAT SOFTWARE ARCHITECTURE IS

The high-level structure and shared understanding of the system

Elements

What major parts exist?

Examples: services, databases, UI, integrations, agents, pipelines.

Relations

How do the parts interact?

Calls, events, shared data, protocols, deployment links, ownership.

Constraints

What rules shape composition?

Security, performance, data boundaries, compliance, reversibility, cost.

- *Architecture is about the **important stuff**. Whatever that is.* **Ralph Johnson, 2003**
- *Architecture is the **decisions that you wish you could get right early** in a project, but that you are not necessarily more likely to get them right than any other.* **Ralph Johnson, 2003**
- *...things that people perceive as **hard to change*** **Martin Fowler, 2003**

Architecture is the important structure developers must share to work safely.

In most successful software projects, the expert developers working on that project have a shared understanding of the system design. This ***shared understanding*** *is called 'architecture.' This understanding includes how the system is divided into components and how the components interact through interfaces. These components are usually composed of smaller components, but the architecture only includes the components and interfaces that are understood by all the developers.*

Ralph Johnson, 2003

WHAT ARCHITECTURE IS NOT

Don’t confuse architecture with ceremony, rank, or diagram theater

Not a big diagram

A diagram helps only if it explains decisions and trade-offs.

Not every detail

Architecture focuses on important structures and hard-to-change decisions.

Not a status symbol

Architect is a responsibility, not a medal for seniority.

Not frozen truth

Architecture evolves as understanding and constraints evolve.

ARCHITECT IS A ROLE, NOT A RANK

Architecture work is about responsibility for coherence and trade-offs

Bad interpretation

“The architect is above the developers.”

Architecture becomes authority without feedback. Decisions are handed down and lose contact with reality.

Better interpretation

“The architect helps the team make and preserve important decisions.”

Architecture work connects product needs, technical constraints, delivery risk, and long-term maintainability.

A good architect is not the person who draws the boxes. It is the person who helps the team understand why the boxes are the way they are.

THE ARCHITECT AS BRIDGE

Like building architecture: understand client needs and engineering constraints

Client / product side

- goals
- business workflows
- usability
- budget
- deadlines
- regulatory constraints

Architecture work

Translate between needs and constraints.

Make trade-offs explicit.

Keep the system coherent as it evolves.

Implementation side

- components
- data
- APIs
- deployment
- security
- performance

QUALITY ATTRIBUTES DRIVE ARCHITECTURE

Architecture is where “how well” requirements become structural decisions

Quality	Architectural pressure	Trade-off
Performance	fewer round-trips, coarser components, caching	may reduce flexibility
Security	layers, isolation, least privilege, validation	adds friction and complexity
Maintainability	fine-grained ownership, clear interfaces	may increase communication overhead
Reliability	redundancy, queues, retries, rollback paths	adds operational cost
Observability	logs, metrics, traces, business telemetry	adds instrumentation work

There is no architecture that maximizes every quality. Design is choosing which qualities matter most for this system.

ARCHITECTURAL STYLES

Reusable ways to organize components and connections

What a style gives

A vocabulary of components, connections, and typical trade-offs.

It helps you recognize recurring system shapes.

What it does not give

A guaranteed solution.

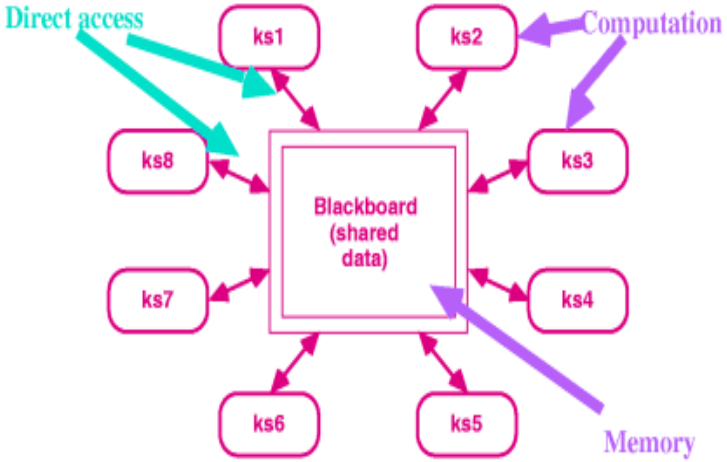
Every style has strengths, weaknesses, and conditions where it becomes painful.

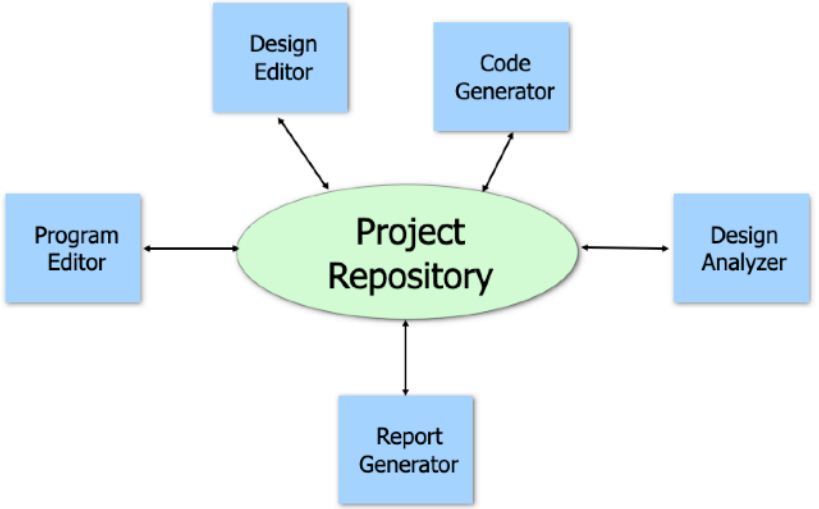
How to choose

Start from forces:
performance, changeability, security, team ownership, data sharing, and deployment constraints.

REPOSITORY STYLE

Many tools/components share one central data store





Good fit: Large shared data model; independent producers/ consumers; centralized backup/security.

Watch out: Central model becomes a compromise; schema changes are expensive; repository can become a bottleneck.

LAYERED ARCHITECTURE

Each layer offers services to the layer above and hides details below

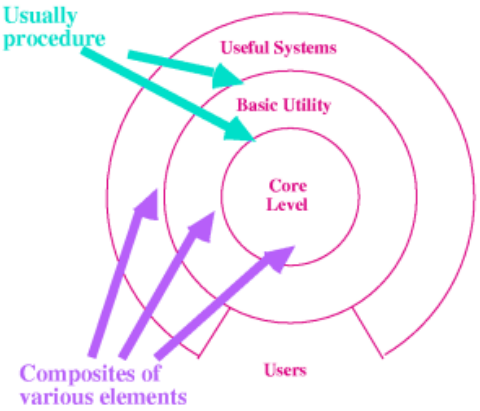
UI / Experience

Application Services

Domain Logic

Data Access

Infrastructure



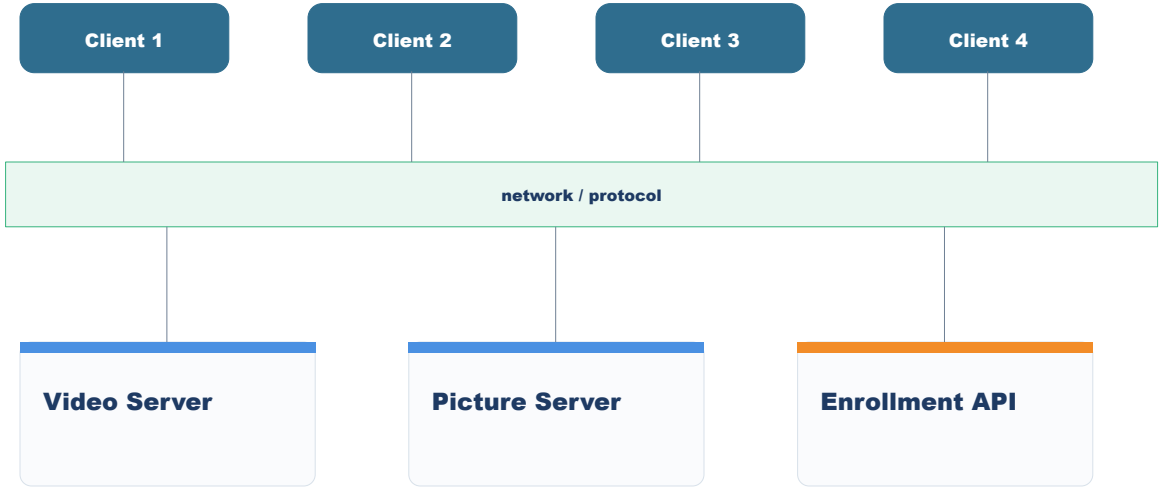
Good fit: Portability, incremental development, security layering, reusable lower layers.

Watch out: Finding the right levels is hard; strict layering can hurt performance.

Design question: Which dependencies are allowed to point upward, downward, or not at all?

CLIENT-SERVER ARCHITECTURE

Clients request services; servers provide specialized capabilities

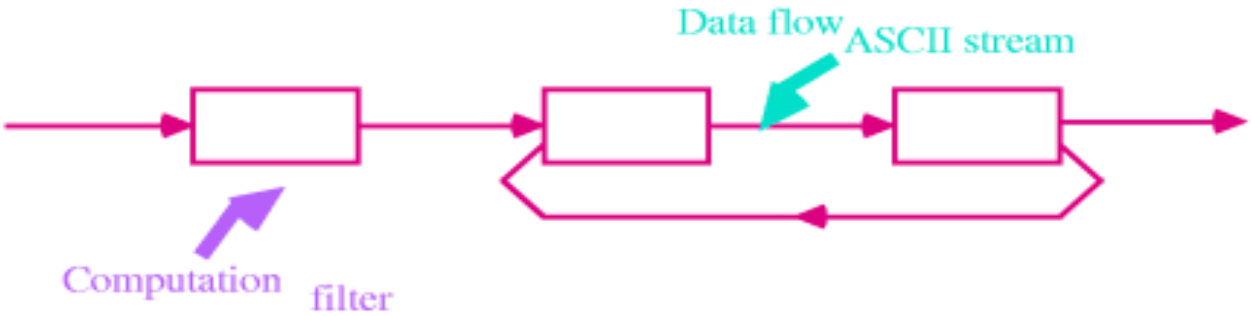


Good fit
Distributed system; new servers can be added; servers can own their data models.

Watch out
Network latency, integration problems, data consistency, and service availability.

PIPES AND FILTERS

A stream of data is transformed step by step



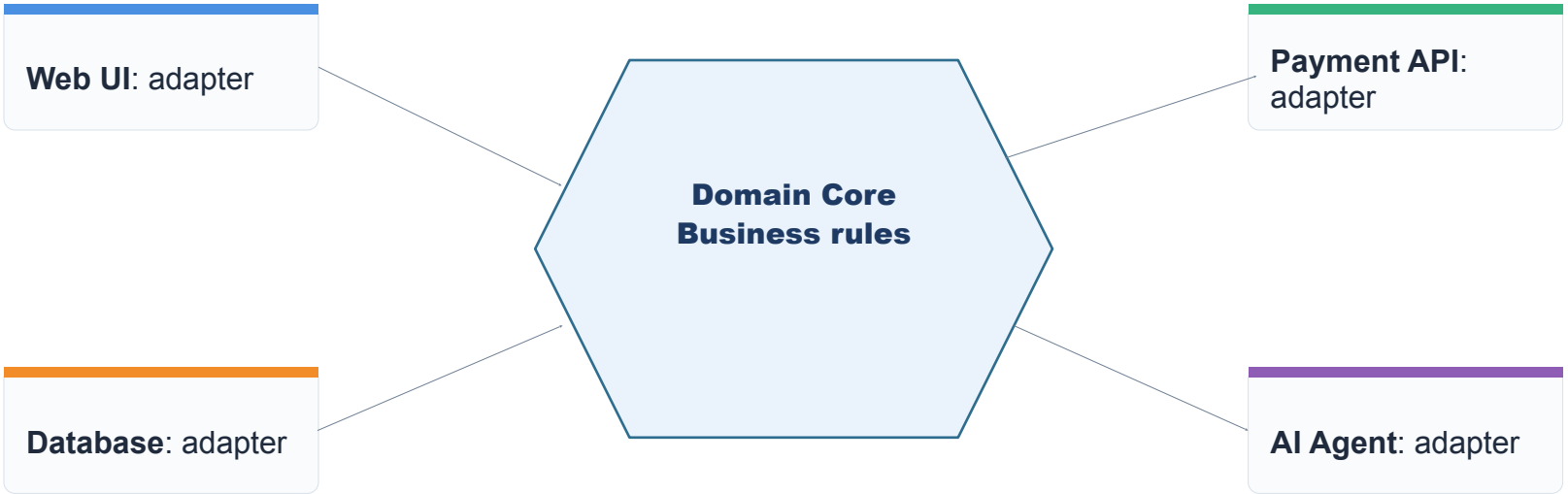
Good fit: Batch processing, compilers, data pipelines, media processing, ETL.

Strength: Each filter is independently understandable and reusable.

Watch out: Not great for highly interactive apps; common data format may create overhead.

MODERN EXTENSION: PORTS AND ADAPTERS

Keep the domain core independent from UI, databases, APIs, and tools

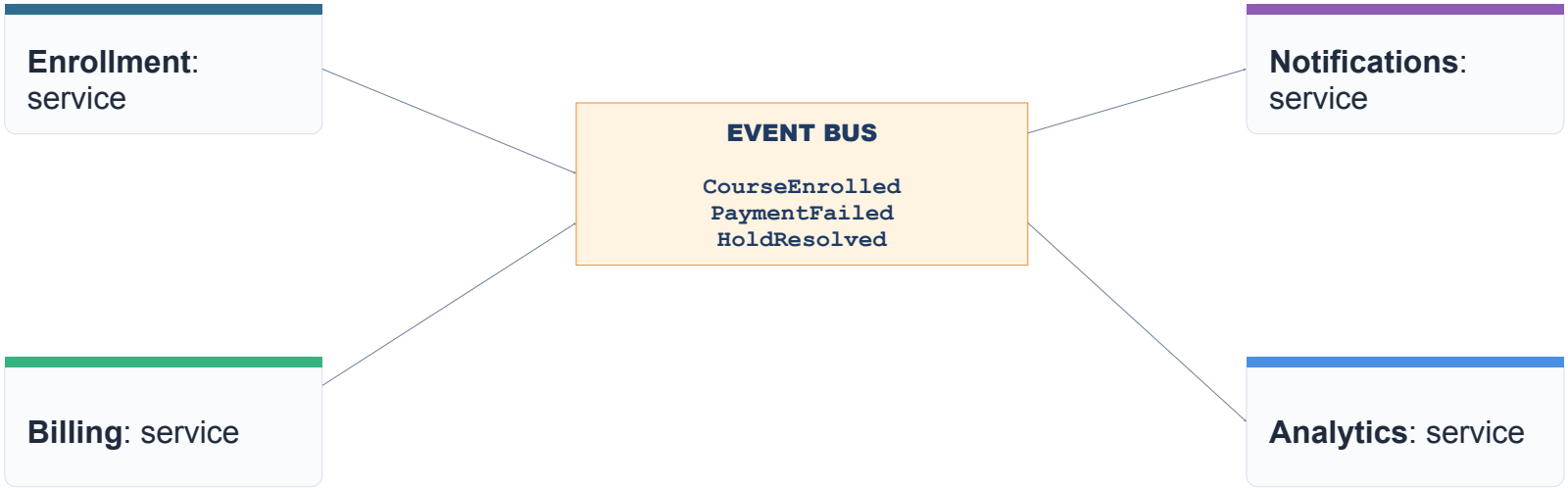


This is a modularity idea at architecture scale: to protect the core from volatile external technology. The center should know the business. The edges should know the technology.

- **Alistair Cockburn, Hexagonal Architecture / Ports and Adapters (2005)**

MODERN EXTENSION: EVENT-DRIVEN SYSTEMS

Components react to facts that happened, instead of calling each other directly



Good fit: Looser coupling, asynchronous workflows, extensibility.

Watch out: Harder tracing, eventual consistency, hidden ordering assumptions.

AI-era relevance: Agents may subscribe to events or trigger tool workflows — governance matters.

CHOOSING AN ARCHITECTURAL STYLE

A style is a trade-off package, not a fashion choice

Large shared data?	**Repository or blackboard might fit.**	But watch centralized schema/control.
Strong security boundary?	**Layering or ports/adapters may help.**	But watch performance and complexity.
Independent teams/services?	**Client-server, microservices, events.**	But watch integration and observability.
Data transformation pipeline?	**Pipes and filters.**	But watch interactive needs and data format overhead.

Ask “what forces shape this system?” before asking “what style do we like?”

AGENT-READY ARCHITECTURE

Design interfaces for machine actors, not only human users

Capability contracts

Expose stable, well-documented operations.

Inputs, outputs, preconditions, postconditions, errors, permissions, rate limits.

Composability

Small, reliable capabilities can be orchestrated by agents.

Huge vague tools force agents to guess.

Invocation reliability

The system must be predictable under repeated tool calls: idempotency, retries, rollback, and audit logs.

Agents make interfaces more important because interfaces become instructions, guardrails, and evidence.

NEW ARCHITECTURE CONCERNS FOR AI AGENTS

The architecture must answer questions old diagrams often ignored

Context
What does the agent see? What is excluded? Who curates it?

Tools
What may it call? With what parameters and permissions?

Memory
What persists across sessions? Who can edit or revoke it?

Verification
What checks prove the task is complete?

Observability
What traces, logs, and artifacts show what happened?

Governance
Who approves high-impact actions? How do we roll back?

These are architectural concerns because they shape safety, reliability, maintainability, and trust.

AI-ERA ARCHITECTURE ANTI-PATTERNS

When generation moves fast, bad structure becomes easier to create

Context spaghetti

Every agent receives too much, too little, or inconsistent context.

Tool soup

Many tools exist, but contracts, permissions, and failure modes are unclear.

Prompt archaeology

Important behavior lives in hidden prompts nobody maintains.

Eval gap

The system has tests for code but no checks for agent behavior.

Silent failure

Agents “complete” tasks without loud evidence of what was verified.

Architecture drift

Generated pieces follow local patterns but violate system boundaries.

The antidote is not “no AI”!

It is explicit architecture: boundaries, contracts, harnesses, observability, and review.



SOFTWARE TESTING

TESTING IS EVIDENCE, NOT CEREMONY

A test is useful only if it can reveal a wrong belief

> ***“Testing can only show the presence of errors, not their absence.”***
>
> **— E. W. Dijkstra**

Not a proof of perfection

A passing suite means: these checks did not find a problem. It does not mean the software is bug-free.

A defect detector

A good test challenges a behavior that might be wrong, especially at boundaries and rare paths.

A feedback tool

Tests help teams learn quickly before users, integration, or production expose the issue.

VERIFICATION AND VALIDATION

Two questions every serious test strategy must answer

Verification

Are we building the product right?

Check that implementation matches the specification, design, contracts, and expected behavior.

Example: the enrollment rule is implemented exactly as specified.

Validation

Are we building the right product?

Check that the software actually satisfies user needs and real requirements.

Example: students can actually build a semester schedule without confusion.

Both matter: a perfectly implemented wrong feature is still a failure.

THE TESTING MINDSET: TRY TO MAKE IT FAIL

Do not only show that it works

Normal cases

Happy paths matter, but they are rarely enough.

Example: student enrolls when all rules are satisfied.

Boundary cases

Values just below, at, and just above a limit.

Example: exactly 0 seats left, exactly max credits.

Error cases

Invalid input, missing data, unavailable service.

Example: payment API down during enrollment.

Rare paths

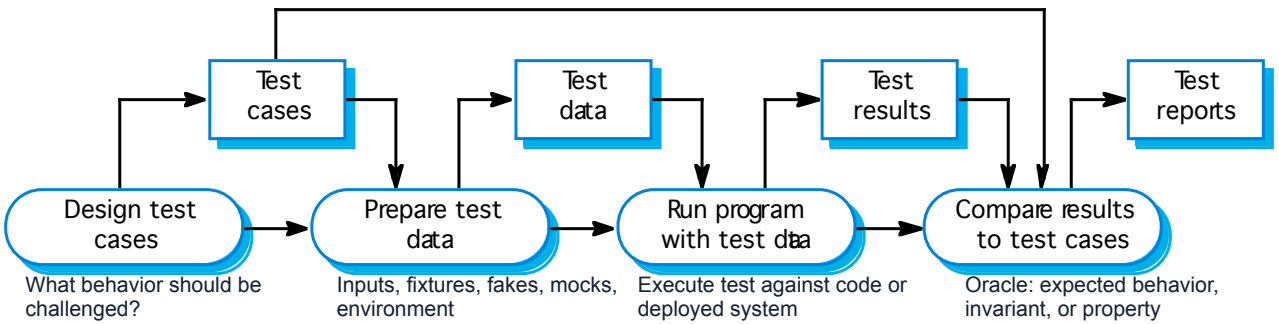
Low-probability paths often hide defects.

Example: two students take last seat.

A good test does not trust the code. It tries to catch it doing something wrong.

THE TESTING PROCESS

A loop from expected behavior to evidence



Report + learn

Failure becomes a fix, a new test, or a better requirement

from **I.Sommerville, SE8**

The important part is the comparison: without an expected behavior, a test is just execution.

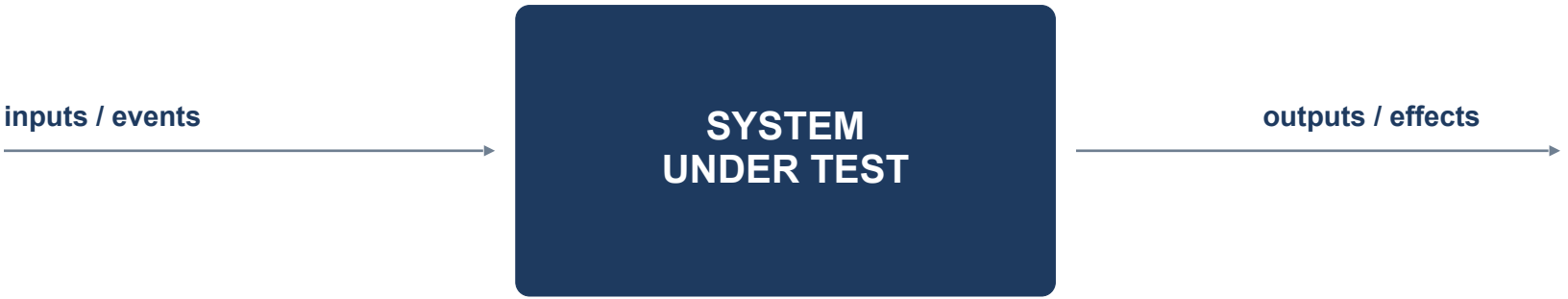
TWO TYPES OF TESTS





BLACK-BOX TESTING

Test behavior from the outside: inputs, events, outputs



Best for

requirements, APIs, user flows, acceptance criteria

You do not need

the internal implementation to design behavior checks

Risk

may miss hidden internal paths or untested branches

EQUIVALENCE PARTITIONING

Choose representatives from classes of similar inputs

Rule

If many inputs should be handled the same way, do not test all of them.

Divide inputs into partitions and test representatives.

Example: credits

Student may enroll only if total credits stay between 20 and 30.

Partitions:
- below 20
- 20–30 valid
- above 30

Pick tests

From each partition choose:
- a normal value
- boundary values
- invalid values that should fail clearly

BOUNDARY VALUES AND SEQUENCES

Most bugs are not in the middle of the road

Boundary values

For range 4–10 inputs, test:

3, 4, 7, 10, 11

Just outside, at edge, typical middle, other edge, just outside.

Sequences

For lists, test:

- empty list
- one element
- several elements
- first / middle / last access
- duplicates if relevant

Course example

Course list results:

- no courses found
- one matching course
- many matching courses
- course at capacity
- waitlist boundary

WHITE-BOX TESTING AND COVERAGE

Look inside the code to see which paths your tests exercise

Statement coverage

Did each line run?

Useful, but weak: a line can execute without checking the right outcome.

Branch coverage

Did each decision go both ways?

Stronger: every if/else should be exercised.

Path coverage

Did important paths through decisions run?

Powerful, but can explode quickly.

Coverage tells you what was executed. It does not prove the assertions were strong.

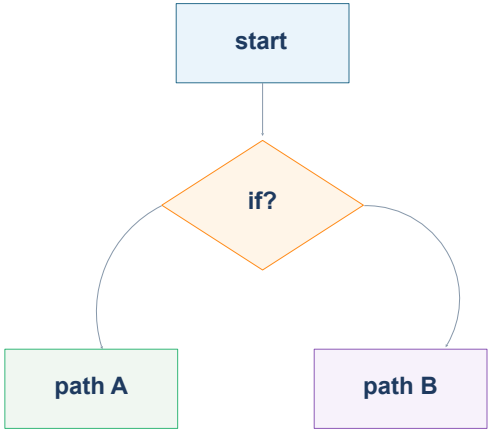
CYCLOMATIC COMPLEXITY: BRANCHY CODE NEEDS MORE TESTS

Every decision point creates more ways the code can behave

Plain meaning

More ifs, loops, and cases mean more independent paths.

More paths mean more tests are needed to gain confidence.



Apply path thinking to critical modules, not every line of the system.



LEVELS OF TESTING

Start small, then test the system as the pieces come together



The loop doesn't stop at release: production behavior is evidence too.

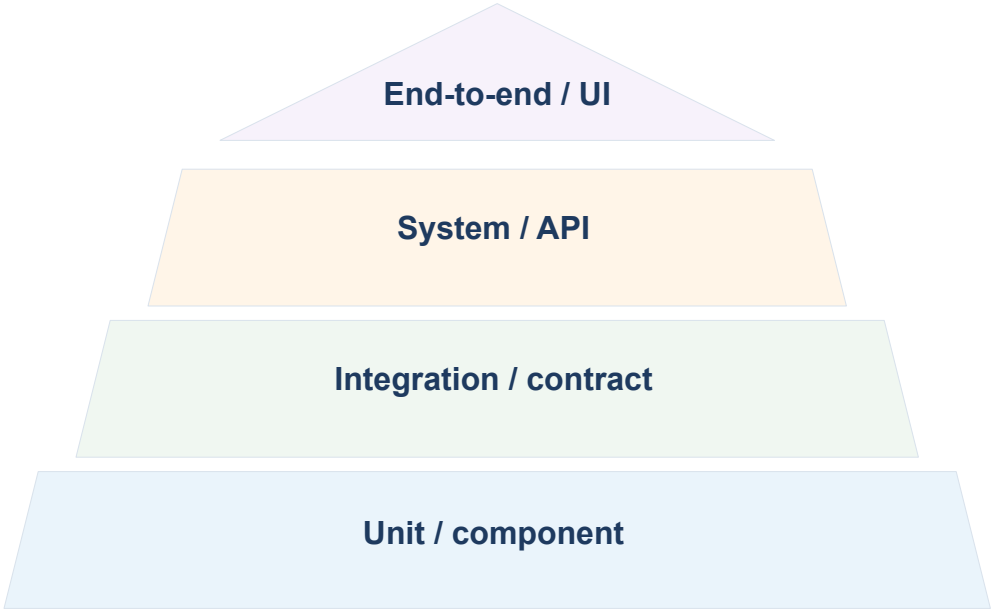
Unit	Integration	System	Acceptance	Production
One class, function, or component	Interfaces between components	Whole application behavior	User goal and requirement satisfaction	Telemetry, alerts, real-world feedback

THE TESTING PYRAMID IS ABOUT FEEDBACK COST

The higher you go, the tests are more realistic; but usually they are also slower and more fragile

Many fast checks

Small tests give quick feedback and isolate causes.



Few realistic checks

System tests catch integration and user-flow failures, but cost more.

Good suites mix levels; they do not rely on one kind of test.

UNIT TESTING

Check a small piece of behavior in isolation

Interface

Do inputs and outputs behave as the contract promises?

Local state

Does temporary data stay consistent while the algorithm runs?

Boundaries

Does it work at limits, just below, and just above?

Paths

Are important decision branches exercised?

Errors

Does it fail clearly and safely?

TEST DOUBLES: STUBS, FAKES, MOCKS

Replace hard-to-control collaborators with controllable ones

Driver / caller

Old term: a small program that calls the unit under test.

Modern view: the test itself drives the behavior.

Stub / fake

A simplified replacement for a dependency.

Example: fake payment service always returns “approved”.

Mock / spy

Checks how a dependency was called.

Example: verify a confirmation email was requested once.

Use test doubles to isolate behavior;
but do not mock away the risk you actually need to test.

INTEGRATION TESTING

Interfaces are where individually correct parts can still fail together

Why unit tests are not enough

A unit can work alone but fail when data crosses a boundary, timing changes, or another component interprets the contract differently.

What integration tests target

- data loss across interfaces
- contract mismatch
- incompatible assumptions
- error propagation
- coordination between components

DO NOT BIG-BANG INTEGRATE

Integrate in small increments so causes remain visible

Big-bang integration

All components are combined first, then the whole program is tested.

Result: many unrelated failures appear at once. Cause isolation becomes hard. Fixing one defect reveals another.

Incremental integration

Add one component or capability at a time.

Run regression checks after each step. Failures are easier to localize and interfaces receive systematic attention.

REGRESSION TESTING

Re-run old checks because new changes can break old behavior

Why it matters

Every change can propagate unintended side effects into behavior that used to work.

What to include

- representative tests across core functions
- tests likely affected by the change
- tests around changed components

Agents can introduce regressions quickly!
A regression suite is a safety net and a feedback channel for the agent.

SMOKE TESTS AND CI

First gate: does the build basically work?

Smoke test

A small set of broad checks that catch show-stopper failures quickly.

If smoke fails, stop deeper testing.

CI pipeline

On every commit or pull request: build, run fast tests, run static checks, report failures.

Agent workflow

An agent should not simply say “done.”

It should produce a patch plus build/test evidence.

WHAT MAKES A GOOD TEST?

A good test maximizes useful information per unit of effort

Fault-revealing

High chance of exposing a real defect or a wrong assumption.

Non-redundant

Adds new information. It is not just another copy of the same easy case.

Redundant tests:

> Test 1: enroll in Software Engineering, seats available.
> Test 2: enroll in Databases, seats available.
> Test 3: enroll in Algorithms, seats available.

Better set:

> Test 1: enrollment succeeds when all rules are satisfied.
> Test 2: enrollment fails when the course is full.
> Test 3: enrollment fails when prerequisite is missing.

Clear oracle

Makes expected behavior explicit. The test fails for the right reason.

Bad test:

> Call `enroll(student, course)` and check that no exception is thrown.

Better test:

```
Given a student has not passed the prerequisite
When the student tries to enroll
Then enrollment is rejected
And the message says: "Missing prerequisite: Data Structures"
And the student is not added to the course
And the seat count does not change
```